

Research article

Mist–fog-assisted real-time emotion recognition using deep transfer learning framework for smart city 4.0

Aditya Ranjan Dalabehera^a, Sujit Beborita^a, Naveen Kumar^b, Dilip Senapati^{a,b,*}^a Department of Computer Science, Ravenshaw University, Cuttack, 753003, Odisha, India^b Department of Computer Science, University of Delhi, New Delhi, 110007, India

ARTICLE INFO

Keywords:

Deep transfer learning
Edge computing
Edge intelligence
Fog computing
Realtime face emotion recognition

ABSTRACT

Face emotion recognition (FER) has found several applications across numerous industries. One such application area lies in the field of human–computer interaction, where it can improve user experiences by allowing systems to adjust based on users' emotional states by leveraging the capabilities of the Internet of Things (IoT) to facilitate more responsive interactions. The increasing demand for precise and immediate emotion analysis across various applications drives the need for FER utilizing deep transfer learning (DTL). In this study, a novel Mist-Fog framework for real-time FER is introduced, which employs a DTL algorithm called MobileNet for quick and easy image classification on mobile and embedded devices. It addresses the challenges of operating deep neural networks (DNN) under limits on power and processing resources. By utilizing depth-wise separable convolutions, the MobileNet algorithm reduces computational complexity while maintaining reasonable accuracy on the FER-2013 dataset. A Fog-based architecture, in which a mist layer comprising edge devices like Raspberry-Pi (RPI), an ARM-based single-board computer, was considered for a smart city scenario. Further, the M/M/1/K queuing system was considered, which follows the First-Come-First-Serve (FCFS) scheduling criteria and a Markov Modulated Poisson Process (MMPP) arrival rate to evaluate the system's performance. The effectiveness of this novel approach towards industrial computer vision paradigms is assessed pertaining to different performance criteria.

1. Introduction

The advent of IoT-based robust systems has revolutionized real-time implementation within contemporary urban environments [1–3]. Additionally, in recent years, the utilization of vision-based processing within the realm of IoT has showcased notable achievements, serving as a critical catalyst for advancing diverse applications and services [4]. Facial expressions are a powerful nonverbal communication tool that reflects a person's emotional state [5]. FER is a vision-based technique that leverages artificial intelligence (AI) to play a crucial role in cognitive science and psychological studies related to facial analysis [6,7]. It involves tracking facial components like eyes, nose, eyebrows, and mouth in images and videos to extract the emotional state of an individual. FER is vital in computer vision and AI, with applications in human–computer interaction [8], affective computing, healthcare monitoring, business and marketing, office and education, smart city monitoring for violence deterrence, and crime investigation [9–11]. FER utilizes IoT technology to enable real-time crowd sentiment monitoring in smart cities and commercial areas. These systems analyze emotional data to enhance public safety measures, facilitate human–computer interactions, and assist in remote patient monitoring. IoT-powered FER helps to identify potential threats, thus improving urban safety, and empowers retailers and

* Corresponding author at: Department of Computer Science, University of Delhi, New Delhi, 110007, India.

E-mail address: senapatidilip@gmail.com (D. Senapati).

<https://doi.org/10.1016/j.iot.2024.101237>

Received 6 February 2024; Received in revised form 25 May 2024; Accepted 26 May 2024

Available online 27 May 2024

2542-6605/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

marketers to understand customer reactions accurately. Integration of IoT with FER enhances the quality of life in diverse sectors. We have developed the generalized IoT-enabled FER for all these diverse sectors. The FER framework consists of face detection, expression analysis, and emotional state classification, which have been addressed using various proposed models [12,13]. In the realm of FER, deep learning (DL) has gained immense popularity due to its high accuracy and efficiency, mainly through pretrained transfer learning neural network models that outperform traditional approaches when applied to large datasets [14]. Fog computing, an innovative paradigm, extends the capabilities of cloud computing to the edge of the network, bringing computing resources closer to end-users [15–17]. This approach reduces latency and improves the quality of service [18]. Furthermore, fog computing offers additional benefits such as enhanced security, privacy, and reliability. To address the computational limitations of resource-constrained IoT devices, fog computing emerges as a promising approach, leveraging nearby fog nodes or remote servers to offload computationally intensive tasks and enable efficient and scalable processing [19,20].

This study introduces a novel Mist-Fog framework for real-time FER by leveraging IoT and fog computing. Unlike previous approaches, which focused solely on FER models, our method integrates IoT devices like RPi at the edge with fog computing capabilities, enhancing efficiency and performance. By integrating IoT devices, FER processing occurs closer to the data source, reducing latency. Fog computing extends cloud capabilities to the network edge, addressing the computational limitations of IoT devices. Our framework utilizes fog nodes for offloading intensive tasks, ensuring efficient real-time FER even on resource-constrained IoT devices. This integration is useful in several practical applications, such as human–computer interaction, healthcare monitoring, and smart city surveillance, providing quick and accurate emotion analysis.

1.1. Contribution

This paper presents a unique IoT-enabled Mist-Fog framework for real-time FER. The recommended method uses MobileNet, a DTL algorithm, to quickly and simply classify images on mobile and embedded devices. It tackles the difficulties of running deep neural networks (DNN) with limited power and processing capabilities. The MobileNet approach maintains respectable accuracy levels while reducing computational costs through depth-wise separable convolutions. For a smart city context, fog-based architecture was taken into consideration. This architecture included a mist layer of edge devices such as the RPi, an ARM-based single-board computer. The M/M/1/K queuing system, which adheres to the FCFS scheduling requirements and has an MMPP arrival rate, was also considered to assess the system's performance. The efficacy of the proposed framework was validated using different performance metrics like precision, recall, F1-score, ROC value, logloss, MCC, and Cohen-Kappa to address the multi-class classification problem. The simulation results for metrics like average time spent in the system for the frames and average number of frames in the system with the offloading factor, along with a trade-off between the offloading factor, average waiting time, and arrival rate, were provided. The optimal cost function was derived for the service rate, which was further used to obtain the trade-off between the number of frames arriving at the fog server and its service rate in a Mist-Fog environment.

1.2. Objective

The main aim of this study is to develop a real-time FER system on an RPi IoT device at the edge and also deploy the trained DTL model with hyperparameter optimization on the edge that would efficiently manage the real-time FER on resource-constrained devices by using a remote server.

1.3. Organization

The remainder of this article is arranged as follows: Section 2 deals with the state-of-the-art review of different articles pertaining to our topic. Section 3 focuses on experimental design with hardware setup and a simulation environment. Section 4 outlines the DTL model design, which further includes subsections like pre-processing, model training and hyperparameter tuning, model deployment, face detection, and feature extraction. These subsections detail the construction of a DTL model for FER, incorporating hyperparameter optimization techniques, and also delve into the practical implementation by deploying the developed model on RPi, highlighting the integration of fog computing to enhance real-time processing capabilities. Moving forward, Section 5 defines the design of a fog computing architecture with a remote control server by utilizing the offloading strategy. This section also introduces an image frame transfer system based on queuing criteria and models frame arrival patterns using MMPP along with the optimal cost model. In Section 6, the results and discussions are furnished, which depict the effectiveness of the overall system's performance and efficiency, provide insights into accomplishing the defined objectives, and highlight the advancements in real-time FER on resource-constrained IoT devices through fog computing. In Section 7, the detailed procedure of complexity analysis is presented. The conclusive summary drawn on the topic has been placed in Section 8, along with the light of future poses.

2. Related work

In this section, we dive into relevant studies for our investigation. Tang et al. [21] introduced a fog computing setup of severe data analysis issues, especially in smart cities. It has key features such as location, low response time, and excellent computing ability. Recently, Vijayakumari and team [22] showcased a new multi-point structure that uses fog computing for quick processing and logging of folks who opt out of vaccines. The system grabs important information like pictures, body temperature, and location data and sends it to a far-away server. The encouraging results highlight the real-world usability of this proposed IoT,

Table 1
Summary of symbols.

Symbol	Description
λ , and μ	Arrival rate and service rate respectively
D	Offloading decision
n	Number of frames in the fog node queue
m	Number of frames being processed in the fog node
k	Maximum capacity of the fog node
$p(\cdot)$	Transition probability
p_{offload}	Probability of offloading a frame
p_{local}	Probability of processing a frame locally
λ_{eff}	Effective arrival rate
L	Average number of frames in the system
W	Average time spent in the system
L_{queue}	The average number of frames in the queue
W_{queue}	The average time spent in the queue
W_{service}	The average time spent during service
ρ	The utilization factor for the fog node
$C(\cdot)$	Total cost function
c_1	Queuing cost of frames
c_2	Waiting cost of frames while availing service
c_3	Waiting cost of frames in queue
c_4	Offloading cost of the frames

fog-booster environment. Sajjad et al. [23] pioneered a cost-effective framework to analyze facial expressions to detect suspicious activities. Leveraging deep learning techniques, their model achieves improved accuracy compared to existing methods. Several researchers [24,25] have explored the application of CNN for FER and achieved reasonable accuracy. Additionally, Hua et al. [26] introduced HERO, leveraging DTL to enhance fast facial expression extraction. Furthermore, Zhai et al. [27] discussed the use of a deep convolutional neural network using NET B and compared it with its variant networks. Several fine-tuned models' architectures were used by Yen et al. [28], where they emphasized the effectiveness of transfer learning models. Liu et al. [29] conducted a study on ensemble CNN, comparing a subnet with ensemble CNN, and comprehensively implemented a CNN variant. Srinivasan and Krishna [30] discuss a modern approach that combines deep learning with transfer learning for fast facial expression detection. This method employs a convolutional neural network (CNN) and surpasses all previous works in terms of speed and accuracy. Adding to this, Shehada et al. [31] presented a novel deep-learning approach utilizing a custom-trained CNN for facial expression recognition, explicitly focusing on addressing challenges visually impaired individuals face. Their model substantially improves recognition accuracy and operates efficiently on edge devices with limited resources. Srinivasan et al. [32] explored the utilization of single-board computers (SBCs) like the Raspberry Pi for real-time emotion recognition. Their evaluation encompassed different deep neural network (DNN) architectures, leveraging neural compute sticks (NCS) such as Intel's Movidius. Meena and their team [33] used deep learning, specifically InceptionV3, for picture-centered sentiment analysis, and they got excellent results on different datasets. This might help study emotional patterns and people's mental health through facial expressions. In another route, Punuri and associates [34] suggested the EfficientNet-XGBoost, a way for recognizing facial expressions using deep learning methods such as CNN. They effectively handled problems like data imbalance and the disappearing gradient issue using analysis and fine-tuning techniques, ultimately obtaining impressive accuracy rates. Abdulazeez and Askar [35] examined reinforcement learning (RL) and deep reinforcement learning (DRL) and checked their use for task delegation in fog computing. Goel and Tiwari [36] looked at resource scheduling methods in fog computing which aimed to improve the quality of service (QoS) in IoT apps. Hosseinzadeh and team [37] thoroughly reviewed task scheduling methods in cloud-fog situations. They evaluated the pros and cons of these methods and also explored using metaheuristic-based strategies. Terefe et al. [38] proposed an algorithm for energy-efficient offloading policies to address the downsides of mobile devices. Their findings indicate a considerable decrease in energy usage compared to the traditional way of offloading when some energy-intensive parts are distributed across the cloud servers.

3. Experimental design

3.1. Experimental setup

To facilitate real-time video analysis, we configured the RPi, an ARM-based single-board computer with its camera module. By capturing live video frames, we utilized them as input for the deployed TensorFlow-Lite model, enabling facial emotion recognition. Moreover, we set up the RPi's inbuilt Wi-Fi module to establish a network connection with the fog node or remote control server. The RPi-4 Model-B has a built-in Wi-Fi module, which supports 2.4 GHz and 5 GHz Wi-Fi frequencies to establish wireless connectivity. This allows for seamless communication between the RPi and external systems. Furthermore, we utilized the Python-GPU environment for DTL training and evaluation and used MATLAB on an AMD Ryzen5 processor for simulation and analysis. The network architecture utilized for FER is MobileNet, a deep transfer learning (DTL) algorithm. MobileNet is specifically chosen for its suitability in mobile and embedded devices due to its lightweight nature and efficient computational capabilities. MobileNet primarily consists of depthwise separable convolutions. In a depthwise separable convolution, each input channel undergoes

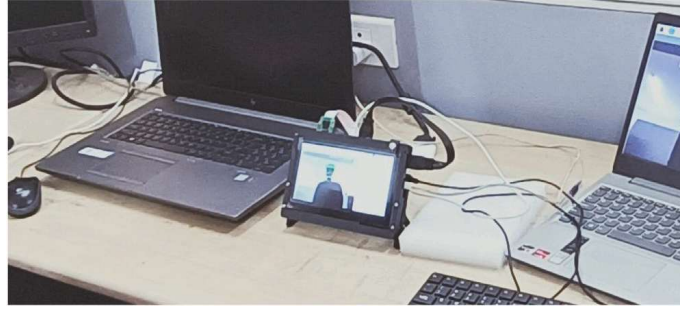


Fig. 1. Experimental setup for the proposed Mist-Fog framework.

convolution with its own kernel, followed by pointwise convolution to linearly combine channels, drastically reducing computations while maintaining performance. This model comprises 89 layers, including 14 Conv2D layers, 27 BatchNormalization, 27 ReLU, 13 DepthwiseConv2D, 4 ZeroPadding2D, 1 GlobalAveragePooling2D, 1 Dropout, and 1 Dense layer. The MobileNet architecture is enhanced by adding a global average pooling layer followed by a dropout layer with a rate of 0.6 and a dense output layer with a softmax activation function, resulting in a final classification layer with seven units, aligning with the seven emotions for classification. Fig. 1 is the representation of the experimental setup for Mist-Fog frameworks. We consider the RPi edge device a Mist node and our system a Fog node remote server and observed that the FPS rate is meager in resource-constrained RPi. So, a Mist-Fog frame offloading approach is indeed to efficiently manage the smart city in the context of real-time image processing.

4. Deep learning model design

4.1. Preprocessing

The DL model utilized in this research was trained on the FER-2013 dataset [39], consisting of labeled grayscale facial images (48×48 pixels) representing seven emotions: anger, disgust, fear, happiness, sadness, surprise, and neutral. To address the dataset imbalance problem, we employed data augmentation methods such as resizing, grayscale conversion, flipping, rotating, and pixel value normalization without performing any over-sampling or under-sampling techniques, resulting in a training set of 28,709 images and a testing set of 7,178 images.

4.2. Model training and hyper-parameter tuning

The MobileNet [40] architecture is employed as the baseline model for the given task with pre-trained weights from ImageNet. Its capabilities are enhanced by adding a global average pooling layer to the base model's output. Subsequently, a dropout layer with a rate of 0.6 is introduced. The output layer consists of a dense layer with 7 units, aligning with the number of classes in the classification task. This layer is further enhanced by applying a softmax activation function, which facilitates the interpretation of the output as probabilistic values [41].

$$\text{Softmax}(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}, \quad (1)$$

where z_j is the logit for class j and K is the number of classes. The training procedure involves designating earlier layers as trainable while keeping the remaining layer weights frozen for fine-tuning. Compilation involves categorical cross-entropy loss, Adam optimizer, and accuracy metrics [42].

$$\text{Categorical cross entropy loss} = - \sum_{i=1}^N y_{true}^{(i)} \log(y_{pred}^{(i)}), \quad (2)$$

where N is the number of training samples, $y_{true}^{(i)}$ is the true label for sample i (one-hot encoded), and $y_{pred}^{(i)}$ is the predicted probability distribution for sample i . Further, we utilized an Adam optimizer [43],

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t,$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t},$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$

$$\theta_t = \theta_{t-1} - \frac{\text{Current learning rate}}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t,$$

where, m_t and v_t are the first and second moment estimates at time t , β_1 and β_2 are the exponential decay rates for the moment estimates, g_t is the gradient at time t , $\hat{m}_t$ and $\hat{v}_t$ are the bias-corrected moment estimates, θ_t is the parameter at time t , and ϵ is a small constant to avoid division by zero. Here, the value of current learning rate is updated by the ReduceLROnPlateau callback.

The ReduceLROnPlateau callback in TensorFlow Keras adjusts the learning rate by a factor of 0.3 when the monitored metric fails to improve for a specified number of epochs. The min-delta = 0.001 parameter determines the minimum improvement required to trigger the learning rate adjustment. The model is trained for 50 epochs with a batch size of 128 using augmented training data. The fine-tune model architecture is summarized as follows:

$$\text{Model output} = \text{Dense}(7, \text{activation} = \text{softmax})(\text{Dropout}(0.6) \\ (\text{Global Average Pooling 2D}(\text{base model output}))). \quad (3)$$

Eq. (3) illustrates the sequential application of operations: global average pooling on the base model's output, followed by dropout regularization with a rate of 0.6, and ultimately a dense layer with 7 units and softmax activation. The resulting output represents the final classification probabilities for the 7 classes. The total parameter counts are 3,236,039, from where, 3,214,151 are trainable and 21,888 are non-trainable.

4.3. Model deployment

After completing the training and evaluation of our DL model, we deployed it on the RPi-4 model-B by utilizing TensorFlow-Lite, a lightweight framework specifically designed for edge devices. The deployment process involved several essential steps. Firstly, we converted our trained model into the TensorFlow-lite format, allowing optimal execution on resource-constrained devices like the RPi. Although this conversion reduced the model's size, its performance remained intact. Next, we seamlessly integrated the converted model into the RPi environment by installing the necessary libraries and dependencies. This integration ensured the smooth execution of TensorFlow-Lite inference on the RPi. Utilizing transfer learning with MobileNet DTL for real-time FER on IoT devices, like RPi, saves computational costs. Leveraging a pre-trained MobileNet model, fine-tuning it for FER, and converting it into a lightweight TensorFlow-lite format reduces memory usage. MobileNet design minimizes parameters and complexity, which is crucial for efficient execution on IoT devices. This approach effectively optimizes resource utilization in IoT-constrained environments, maintaining satisfactory performance levels.

Transfer learning leverages pre-trained models to extract meaningful features from data, reducing computational costs by avoiding training deep neural networks from scratch. Training with pre-trained weights, requiring fewer iterations for satisfactory performance, minimizes the need for large labeled datasets, as it can leverage knowledge from related tasks or domains, thereby enhancing training efficiency. Further, TensorFlow Lite aids in reducing computational requirements while maintaining performance. TF.Lite models are optimized for mobile and edge devices by including only essential components, resulting in minimal metadata and smaller file sizes compared to Keras models in H5 format. This design ensures efficient storage and inference. Thus, transfer learning, with the help of TensorFlow Lite, enables more efficient resource allocation, ensuring optimal performance on constrained devices.

4.3.1. Face detection

We integrate the Haar-Cascade algorithm for face detection in our deployment pipeline so that the model receives accurate input data by detecting and extracting faces from the video frames captured by the RPi camera module. This enables us to focus on recognizing facial emotions within the detected regions. The Haar-Cascade classifier is a specific implementation of the Viola-Jones algorithm. We chose to utilize the Haar-Cascade algorithm over the broader Viola-Jones algorithm for face detection in our deployment pipeline for its efficiency, accuracy, and suitability for real-time applications on resource-constrained IoT devices like RPi [44,45].

4.3.2. Feature extraction

Each video frame captured by the RPi camera module underwent processing by the deployed lightweight model to extract emotions accurately. By leveraging the power of TensorFlow-Lite, we achieved real-time FER on the RPi, enhancing the system capabilities on the edge. Fig. 2 provides an overview of the proposed architecture, presenting the hierarchy of layers with specific functionalities.

Fig. 3 offers a sequence diagram for the proposed FER system. A sequence diagram is crucial for visually displaying the interactions between various system components. The four essential components contributing to this task offloading scenario are the IoT layer responsible for image acquisition, the mist computing layer, the fog layer, and the cloud layer. The IoT device layer acts as the first stage of face image acquisition. Before task offloading is used, these images are analyzed locally to extract pertinent emotional information regarding a person. The mist computing layer is an intermediate between the IoT and Fog-Cloud layers. Light-weight computational activities are carried out in this layer to pre-process the acquired images. This pre-processing entails noise reduction, feature extraction, and image filtering. The mist computing layer minimizes the quantity of data sent to upper layers, lowering latency and improving system performance overall by handling these activities locally. The next layer is the fog computing layer, in which more complicated computing operations are carried out using the resources in fog nodes or edge devices. The fog

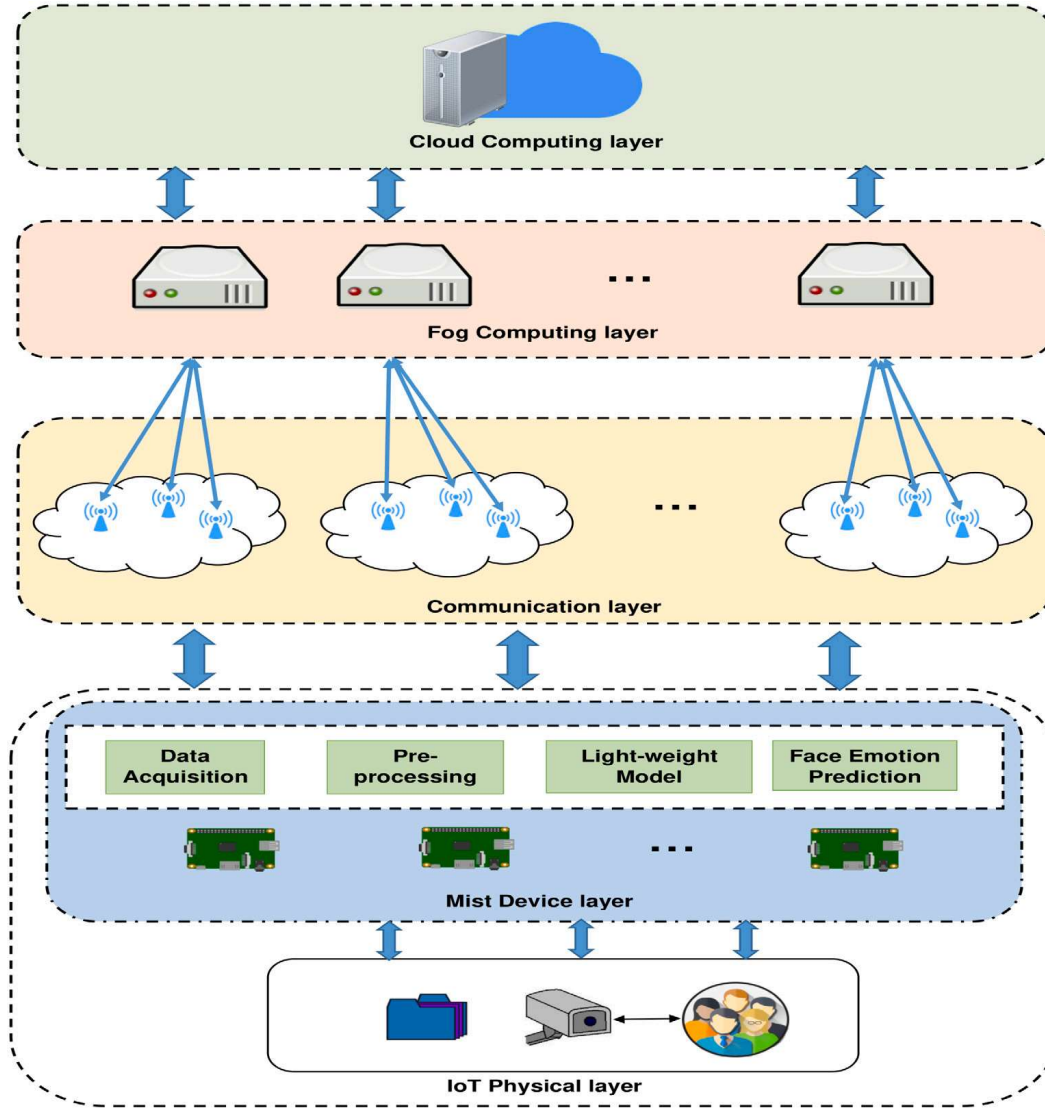


Fig. 2. Fog computing infrastructure for smart city 4.0.

layer has more powerful computing capabilities than the mist computing layer, enabling advanced image processing methods like object detection, recognition, and tracking. To lessen the stress of local processing, some computationally intensive tasks are given to the fog nodes or even redirected to the cloud layer from this layer. Finally, the cloud layer acts as the topmost layer of the system structure. The cloud layer comprises vital data centers with high processing capacity. Tasks that are complex for the fog layer or require a lot of storage are transferred to the cloud layer. Depending on the system's needs, results and outputs from the cloud layer are subsequently sent back to the fog layer or even directly to the IoT layer. The sequence diagram shows the synchronized movement of data and tasks across the system layers by interacting with these four components, focusing on the task offloading process. These layers work together to provide a seamless ecosystem that enhances computing effectiveness, lowers latency, and allows for complex image processing in the context of task offloading.

As illustrated in Fig. 4, the fine-tuned model architecture for FER includes layers and operations involved in emotion classification. Fig. 5 comprises various procedural workflows starting from the preprocessing of the dataset to the generation of a lightweight model. The lightweight model obtained from Fig. 5 is subsequently loaded into the edge setup, as depicted in Fig. 6. This setup demonstrates the real-time edge environment, including the RPi and camera module, and shows the entire process, from loading the lightweight model and real-time acquisition to performing real-time detection, inference, and frame offloading. The edge devices acquire raw input via the RPi camera module, and frames can be offloaded to the fog node control server. The applications in IoT-aided environments include healthcare monitoring, public places monitoring for crime deterrence, and business marketing.

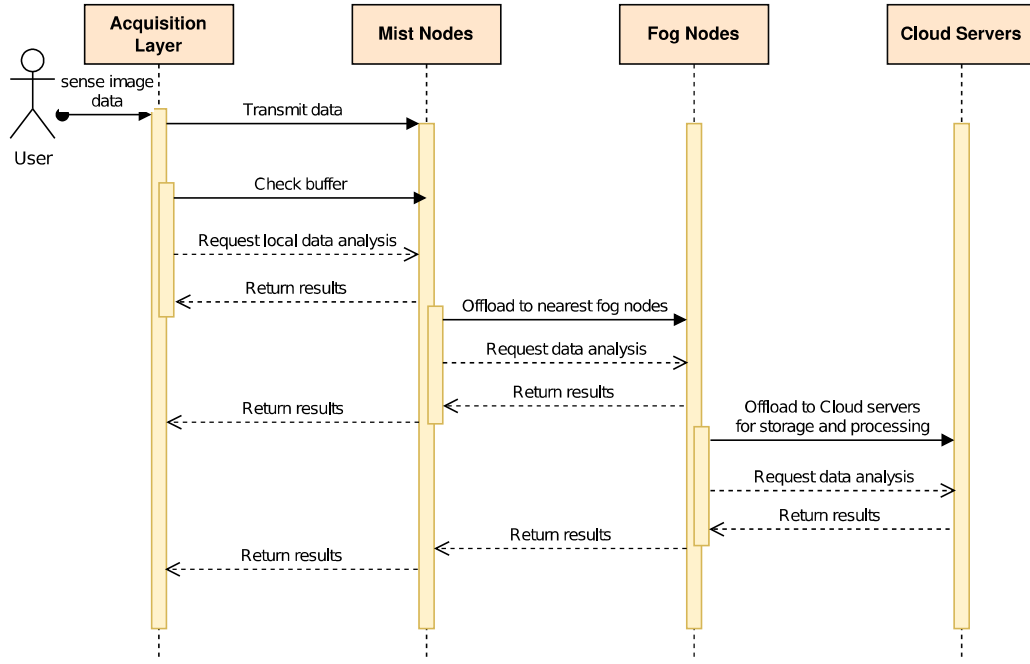


Fig. 3. Sequence diagram for proposed method.

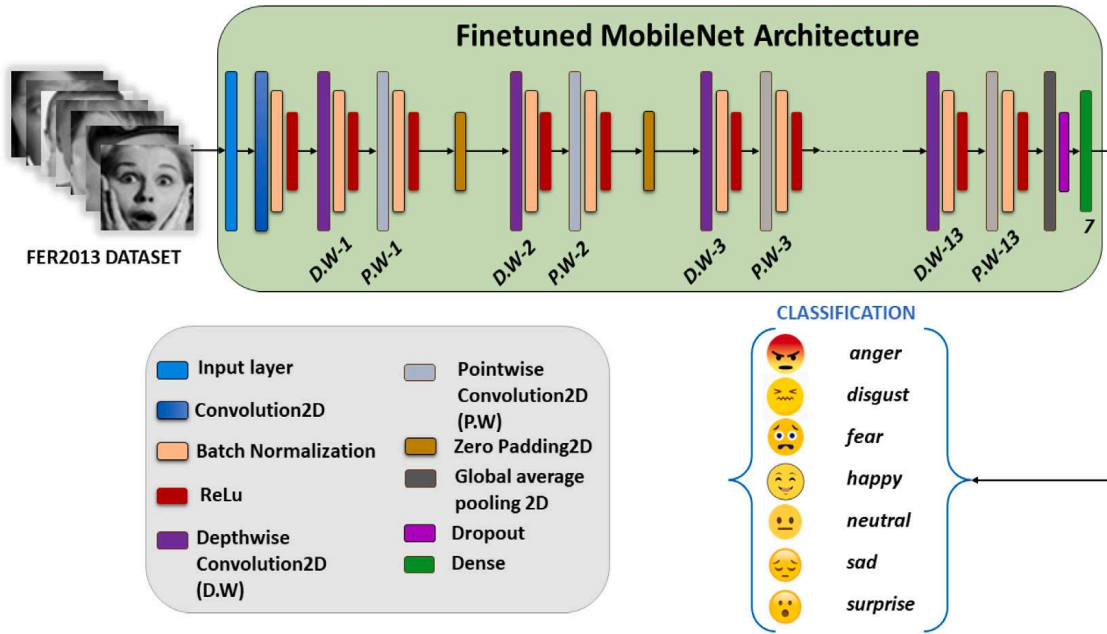


Fig. 4. Schema illustrating the fine-tuning model architecture for FER, encompassing layers and operations involved in emotion classification.

5. Offloading strategy

The extensive growth in Industrial Internet of Things (IIoT) applications has tremendously increased the demands for low latency and resource-sensitive computing to accomplish critical industrial automation [46]. The RPi mist node continuously captures video frames through its physical layer camera module. We enabled real-time FER tasks on the RPi mist layer, with the added capability of offloading frames to the fog node for processing when necessary [47–49]. The fog layer provides enhanced computing capabilities compared to the mist layer, facilitating advanced image processing tasks. Therefore, in this scenario, the computer system serves as the fog node remote server at the control station, handling computationally intensive tasks with a vast amount of data. The arrival process follows an MMPP model [50]. This means frame arrivals are considered to conform to a Poisson distribution with a rate of λ . Additionally, we assume that frame service time at the fog node adheres to an exponential distribution with a rate of μ .

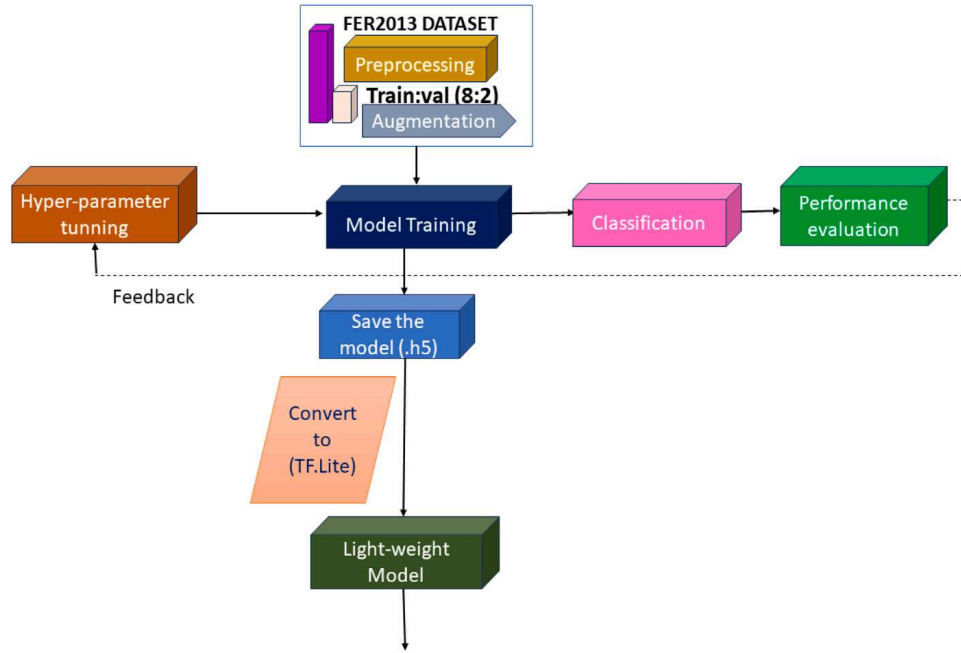


Fig. 5. Schema illustrating the procedural workflow for FER.

Furthermore, we consider the fog node to be a single-server system capable of processing a maximum of k frames simultaneously. Consequently, the queuing model at the fog node operates on an M/M/1/ k system with FCFS queue scheduling [51]. The arrival rate (λ) represents the frequency of frames captured by the RPi camera module in frames per second (FPS). The service rate (μ) indicates the speed at which the fog node can process frames, also measured in FPS. The maximum concurrent processing (k) is the maximum number of frames the fog node can handle simultaneously. The offloading probability (p_{offload}) determines the likelihood of transferring a frame to the fog node for processing.

The offloading probability (p_{offload}) is a metric used to decide whether to send a frame to the fog node or process it locally on the RPi mist node [52,53]. It is calculated using the equation:

$$p_{\text{offload}} = \max \left\{ 0, \frac{\lambda}{\mu + \lambda(k-1)} \right\}. \quad (4)$$

The offloading probability will be capped at the maximum value between 0 and $\frac{\lambda}{\mu + \lambda(k-1)}$. Maximizing the offloading probability implies a preference for offloading frames to the fog node for processing. It indicates that the system aims to leverage the fog node's processing capabilities to handle as many frames as possible, optimizing resource utilization and potentially improving processing efficiency. The local processing probability (p_{local}) refers to the probability of processing a frame locally on the mist node:

$$p_{\text{local}} = 1 - p_{\text{offload}}. \quad (5)$$

5.1. Analytical case study

We represent the system state as (n, m) , where n denotes the number of frames in the fog node's queue, while m represents the number of frames being processed at the fog node (i.e., $0 \leq n \leq k$ and $0 \leq m \leq k$).

The transition probability matrix (TPM), can be defined as follows:

$$\begin{pmatrix} p(0,0) & p(0,1) & p(0,2) & \dots & p(0,k) \\ p(1,0) & p(1,1) & p(1,2) & \dots & p(1,k) \\ p(2,0) & p(2,1) & p(2,2) & \dots & p(2,k) \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ p(k,0) & p(k,1) & p(k,2) & \dots & p(k,k) \end{pmatrix}$$

There are 4 different scenarios.

Case 1: $p(0,0)$ corresponds to when the fog node is idle, with no frames being processed or in the queue. The transition probability $p(0,0)$ is equal to the probability of processing the frame locally (p_{local}).

$$p(0,0) = p_{\text{local}}. \quad (6)$$

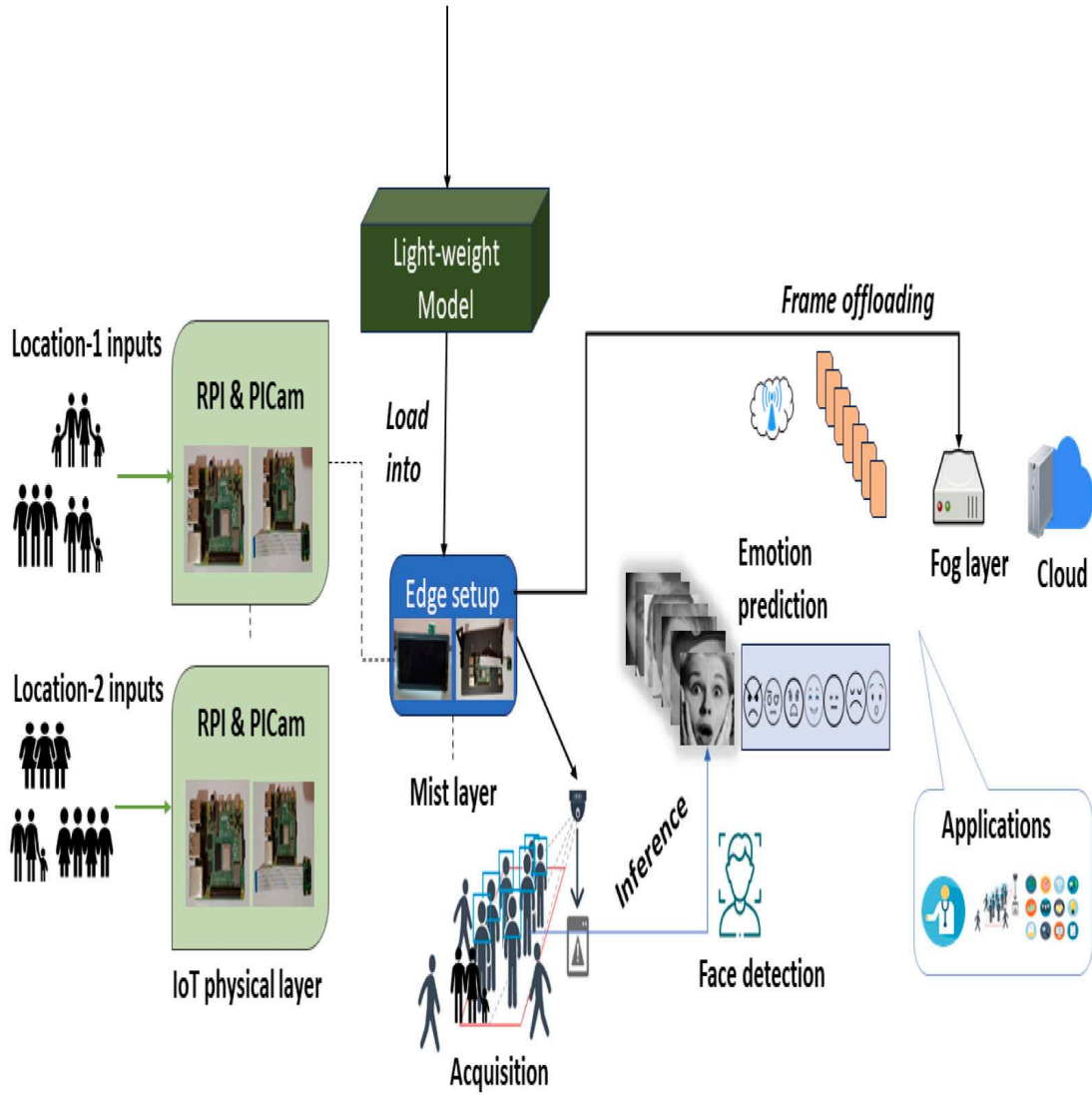


Fig. 6. Global architecture schema showing the procedural workflow from acquisition of face images to prediction of human sentiment.

Case 2: $p(0, m)$ represents the fog node being idle while m frames are being processed. $p(0, m)$ is calculated using the complement of local processing probability and offloading m frames to the fog node ($p_{offload}^m$).

$$p(0, m) = (1 - p_{local}) \times p_{offload}^m. \quad (7)$$

Case 3: $p(n, m)$, where $n > 0$ and $m < k$. In this case, there are n frames in the queue, and m frames are being processed. $p(n, m)$ takes into consideration the probability of offloading $(n + m)$ frames with the complement of both local processing and offloading probability.

$$p(n, m) = (1 - p_{local}) \times p_{offload}^{n+m} \times (1 - p_{offload}), \quad (8)$$

where, $n > 0$ and $m < k$.

Case 4: $p(n, k)$ occurs when there are n frames in the queue, and the fog node is processing k frames (reaching its processing capacity). The transition probability $p(n, k)$ is given by the complement of the probability of local processing ($1 - p_{local}$) multiplied by the probability of offloading $(n + k)$ frames to the fog node ($p_{offload}^{n+k}$). The sum of probabilities for each state (n, m) within the transition probability matrix should equal 1.

$$p(n, k) = (1 - p_{local}) \times p_{offload}^{n+k}. \quad (9)$$

5.2. Offloading decision and queuing performance metrics

The offloading decision (D) is based on the state of the mist node queue and represents the decision to offload or process locally. D is 0 for local processing, and 1 means the mist node queue is full, indicating for offloading.

$$D = \begin{cases} 1, & \text{Mist node queue is full,} \\ 0, & \text{otherwise.} \end{cases}$$

Using the TPM, the average number of frames in the queue (L_{queue}) is:

$$L_{queue} = \sum_{n=0}^k \sum_{m=0}^k n \cdot p(n, m). \quad (10)$$

By utilizing the M/M/1/k queuing model, according to Little's law the average time spent in the fog node queue (W_{queue}) is:

$$W_{queue} = \frac{L_{queue}}{\lambda}. \quad (11)$$

The effective arrival rate (λ_{eff}) takes into account both the offloading probability ($p_{offload}$) and the arrival rate (λ).

$$\lambda_{eff} = \lambda \cdot (1 - p_{local}) \cdot p_{offload}. \quad (12)$$

The average time spent during service ($W_{service}$) is:

$$W_{service} = \frac{1}{\mu - \lambda_{eff}}. \quad (13)$$

The average time spent in the system (W) can be obtained by adding the time spent in the fog node queue (W_{queue}) and the time spent during service ($W_{service}$) [53]:

$$W = W_{queue} + W_{service}. \quad (14)$$

By substituting the values of W_{queue} and λ_{eff} ,

$$W = \frac{L_{queue}}{\lambda} + \frac{1}{\mu - \lambda \cdot (1 - p_{local}) \cdot p_{offload}}. \quad (15)$$

Finally, by multiplying the arrival rate (λ) by the average time spent in the system (W), we can calculate the average number of frames in the system (L) at the fog node (Little's law) $L = \lambda \cdot W$ [53]. Utilization factor for the fog node based on the average number of frames being processed (m) and its maximum processing capacity (k). The utilization factor ρ is defines as $\rho = m/k$.

This provides valuable insights into how effectively the fog node is being utilized. A high utilization factor might indicate that the fog node is operating close to its maximum capacity, while a low utilization factor might suggest that the fog node has spare capacity.

5.3. Cost model

In this subsection, we formulate the expected cost pertaining to processing of the frames over the fog servers. We estimate the best overall expected cost over the service rate μ and arrival rate of frames λ by creating an anticipated cost function designated as $C(\lambda, \mu)$. We introduce the following cost elements here for definition and analysis viz., c_1 representing the queuing cost of frames, c_2 denoting the waiting cost of frames while availing service, c_3 gives the waiting cost of frames in queue, and c_4 provides offloading cost of the frames. The expression for the total expected cost function per unit time can be constructed as follows using the definitions of each cost parameter and the corresponding system performance measurements that are provided [1,4,19,46,54,55]:

$$C(\lambda, \mu) = c_1 L_{queue} + c_2 W_{service} + c_3 W_{queue} + c_4 p_{offload}. \quad (16)$$

$$C(\lambda, \mu) = c_1 (L - m) + c_2 \left(\frac{1}{\mu - \lambda_{eff}} \right) + c_3 \left(\frac{L_{queue}}{\lambda} \right) + c_4 \left[\frac{\lambda}{\mu + \lambda(k-1)} \right]. \quad (17)$$

Algorithm 1 DTL training and evaluation

- 1: **Inputs:** FER-2013 dataset
 - 2: **preprocess** the FER2013 dataset using libraries TensorFlow , Keras
 - 3: **Augment** the dataset for imbalance problem
 - 4: **Initialize** Mobile-Net DTL model
 - 5: **Tune hyperparameters** for model training
 - 6: **Evaluate** the model
 - 7: **Convert** the fine-tuned model into Light-weight format
 - 8: **Output:** Light-weight model
-

Algorithm 2 Real-time Emotion Recognition on Raspberry Pi

```

1: Input: Light- weight model generated by algorithm-1
2: Initialize the Raspberry Pi camera module
3: while video not end do
4:   Capture a real-time frame from video in FPS via RPi cam
5:   Preprocess the frame to extract faces using OpenCV
6:   Pass the faces through the trained model to predict emotions
7:   if (Mist_Queue = Full) then
8:     Offload the frames to the fog node for processing
9:   else
10:    Process the frame in the RPi
11:   end if
12:   Output: predicted emotion in each frame of the video
13: end while

```

Algorithm 3 Offloading Decision and Queuing Performance Evaluation

```

1: video (frames,  $\lambda$ ,  $\mu$ ,  $m$ ,  $n$ ,  $k$ )
2: Input parameters:
3:   frames - instances of video
4:    $\lambda$  - arrival rate
5:    $\mu$  - service rate
6:    $m$  - number of frames being processed in the fog node
7:    $n$  - number of frames in the fog node
8:    $k$  - capacity of the fog node
9: if (Mist_Queue != Full) then
10:   Process frame locally in RPi //  $D = 0$ 
11: else
12:   Offload frame to Fog Node //  $D = 1$ 
13: end if
14:  $p_{offload} = \max \left\{ 0, \frac{\lambda}{\mu + \lambda(k-1)} \right\}$  // offloading probability
15:  $p_{local} = 1 - p_{offload}$  // local processing probability
16:  $\lambda_{eff} = \lambda \cdot (1 - p_{local}) \cdot p_{offload}$  // effective arrival rate
17: Compute Transition probability matrix  $p$ .
18:  $p(0, 0) = p_{local}$ 
19:  $p(0, m) = (1 - p_{local}) \cdot (p_{offload})^m$  for  $m < k$ 
20:  $p(n, m) = (1 - p_{local}) \cdot (p_{offload})^{n+m} \cdot (1 - p_{offload})$  for  $n > 0$  and  $m < k$ 
21:  $p(n, k) = (1 - p_{local}) \cdot (p_{offload})^{n+k}$  for  $n > 0$ 
22:  $L_{queue} = \sum_{n=0}^k \sum_{m=0}^k n \cdot p(n, m)$  //average queue length
23:  $W_{queue} = \frac{L_{queue}}{\lambda}$  //average waiting time in the queue
24:  $W_{service} = \frac{1}{\mu - \lambda_{eff}}$  // average waiting time for service
25:  $W = W_{queue} + W_{service}$  // average time spent in the system
26:  $L = \lambda \cdot W$  // average number of frames in the system
27:  $C(\lambda, \mu) = c_1 \cdot L_{queue} + c_2 \cdot W_{service} + c_3 \cdot W_{queue} + c_4 \cdot p_{offload}$  //model cost
28: return cost  $C(\lambda, \mu)$ 

```

6. Results and discussions**6.1. Deep learning performance evaluation**

In this study, we conducted a performance analysis of the proposed architecture over the FER-2013 image dataset. The classification performance metrics [56] are presented in Table 2, which include precision, recall, and F1-score for each class (0 to 6). The accuracy of the model on the dataset is reported as 0.63. Additionally, the macro average and weighted average values for precision, recall, and F1-score are provided. In our analysis of class-specific metrics, we found that class 3 had the highest precision, recall, and F1-score (0.81, 0.84, and 0.83, respectively), while class 2 had the lowest values (0.47, 0.49, and 0.48, respectively). Intermediate performance metrics were observed for classes 0, 1, 4, 5, and 6. The model's overall accuracy was 0.63, indicating correct predictions for 63% of instances. The macro average values for precision, recall, and F1-score were 0.63, 0.60, and 0.61, respectively, suggesting a balanced performance across classes. The weighted average values for precision, recall, and F1-score were

Table 2
Classification performance metrics.

Class	Precision	Recall	F1-score
angry	0.58	0.54	0.56
disgust	0.64	0.48	0.55
fear	0.47	0.49	0.48
happy	0.81	0.84	0.83
neutral	0.59	0.59	0.59
sad	0.51	0.54	0.53
surprise	0.76	0.72	0.74
Accuracy	–	–	0.63
Macro Avg	0.63	0.60	0.61
Weighted Avg	0.64	0.63	0.63

Table 3
One-vs-One and One-vs-Rest ROC-AUC Scores.

Approach	ROC-AUC Score
One-vs-One (macro)	0.879855
One-vs-One (weighted by prevalence)	0.880584
One-vs-Rest (macro)	0.879855
One-vs-Rest (weighted by prevalence)	0.880584

Table 4
Loss and correlation metrics.

Metric	Value
Log Loss	2.70195
Cohen's Kappa	0.55668
MCC	0.55678

0.64, 0.63, and 0.63, respectively, considering the class imbalance in the dataset. Moreover, [Table 3](#) presents the ROC-AUC scores for the One-vs-One and One-vs-Rest approaches. The scores were calculated using both the macro and prevalence-weighted schemes. Notably, the ROC AUC scores for all approaches were consistently high, with values around 0.88. This suggests that the model exhibits a robust ability to distinguish between different classes.

Additionally, [Table 4](#) provides further performance metrics for the model. The log loss value of the model is reported as 2.70. Cohen's Kappa, a metric evaluating the agreement between predicted and actual class labels, yielded a value of 0.56. Similarly, the Matthews Correlation Coefficient (MCC), which considers true positives, true negatives, false positives, and false negatives, also achieved a value of 0.56.

Furthermore, accuracy and loss curves were utilized to monitor the model's progress and improvement. In [Fig. 7](#), the accuracy curve demonstrated the gradual increase in the model accuracy while training the train and test set, and we observed after epochs 25 the model did not improve its test accuracy due to the more imbalance nature of the dataset. While the loss curve in [Fig. 8](#) depicted the corresponding decrease in the model loss function. Furthermore, a confusion matrix in [Fig. 9](#) was employed to comprehensively understand the model's prediction performance for each class. Additionally, ROC curves were generated in [Fig. 10](#) to evaluate the performance both on a class-wise basis and overall in [Fig. 11](#). These curves provided insights into the trade-off between the true positive rate and the false positive rate at different threshold settings. Higher ROC-AUC scores were indicative of better classification performance. Precision–recall curves were also employed to assess the model's precision and recall at various classification thresholds, both for individual classes in [Fig. 12](#). Higher precision–recall curves indicated superior classification performance. The model showcased moderate to good performance across different classes, as evidenced by the balanced average metrics and high ROC-AUC scores. [Fig. 18](#) showcases real-time FER test data results and shows how well the model performs in difficult lighting conditions to detect emotion in night vision scenarios.

FER-2013 is a large and well-known benchmark dataset. However, the dataset suffers from class imbalance, a common issue where certain emotion categories have significantly fewer samples than others. To mitigate this problem, we applied oversampling techniques, augmenting the training set through methods like resizing, grayscale conversion, flipping, rotating, and pixel value normalization. As a result, the dataset achieved a more balanced and equitable representation across emotion classes. Specifically, the post-oversampling dataset contained 47,626 images distributed across 7 classes for training, while the validation dataset comprised 11,900 images. The distribution of samples in the training dataset after oversampling is as follows: 'angry': 7192, 'disgust': 4486, 'fear': 7192, 'happy': 7192, 'neutral': 7192, 'sad': 7192, 'surprise': 7180. Furthermore, we enhanced the number of samples in the disgusting class, which was very small in number, to 5607 samples, and all others remained the same as an original dataset for disgust label balancing. The accuracy and loss curves for training and validation for label balancing and oversampling techniques are depicted in [Figs. 13](#) and [14](#), respectively. Both methods were trained for a small number of training epochs (11 and 7, respectively)

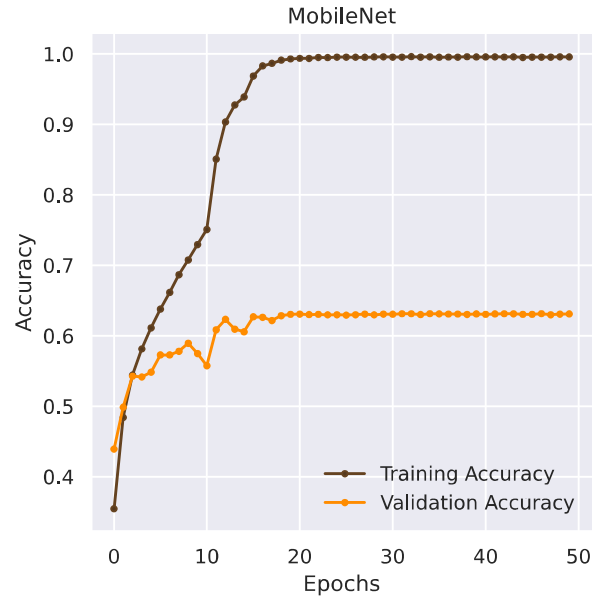


Fig. 7. Comparison of training and validation accuracy for varying number of epochs.

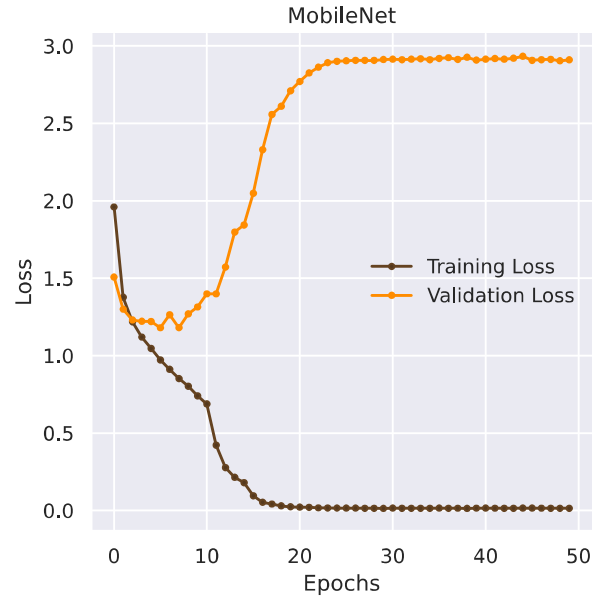


Fig. 8. Comparison of training and validation loss for varying number of epochs.

due to computational constraints but showed reasonable upward trends in terms of accuracy. The model achieved a peak accuracy of 66.58% for disgust labeling and 67.01% for equitable oversampling techniques.

6.2. Performance analysis of the system and observations

In this section, we present the results of the simulation experiments using the proposed mathematical model. We evaluate three aspects: the average number of frames in the system (L), the average time spent in the system (W), and the average waiting time (W_{queue}). Each aspect is examined under various parameter settings, including different arrival rates (λ), offloading factors (representing the proportion of frames offloaded from 0 to 1), and local processing probabilities (P_{local}).

6.2.1. Average number of frames in the system (L)

The average number of frames in the system (L) shows exponential growth based on system parameters. Higher arrival rates (λ) and increased offloading factor result in more frames in the system, as shown in Fig. 21. The relationship between the average

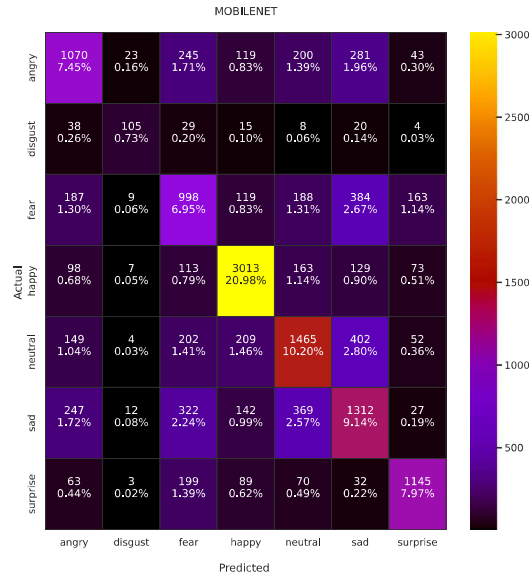


Fig. 9. Confusion matrix for MobileNet.

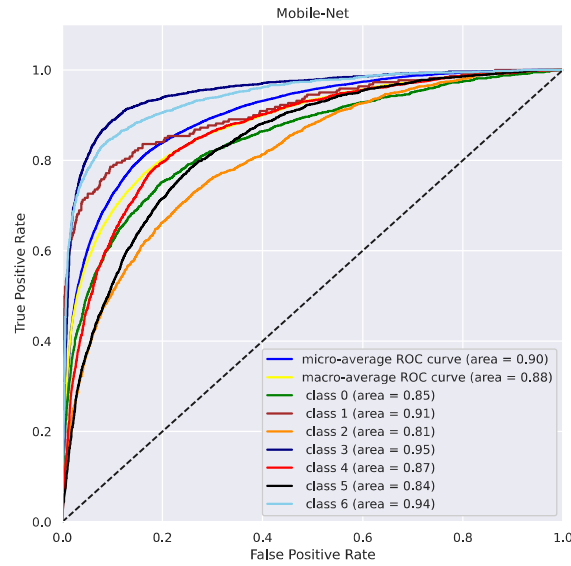


Fig. 10. Class-wise comparison of ROC curves for MobileNet.

number of frames and the offloading factor is depicted, indicating that a higher offloading factor leads to a larger number of frames in the system. Additionally, higher arrival rates contribute to increased workload and more frames in the system.

6.2.2. Average time spent in the system (W)

Fig. 22 shows the exponential growth of the average time spent in the system (W) when arrival rates (λ) and offloading factors change. Increasing arrival rates lead to higher W , indicating longer frame waiting times. Similarly, a higher offloading factor at a constant arrival rate leads to a greater W due to more offloaded frames.

6.2.3. Average waiting time (W_{queue})

The average waiting time W_{queue} focuses on the time frames spend waiting in the queue before being processed. Fig. 23 illustrates the average waiting time across various offloading factors and arrival rates. It is evident that higher offloading factors and arrival rates lead to extended waiting times, suggesting that increased workload and offloading operations can cause delays in frame processing, which may have an impact on real-time applications.

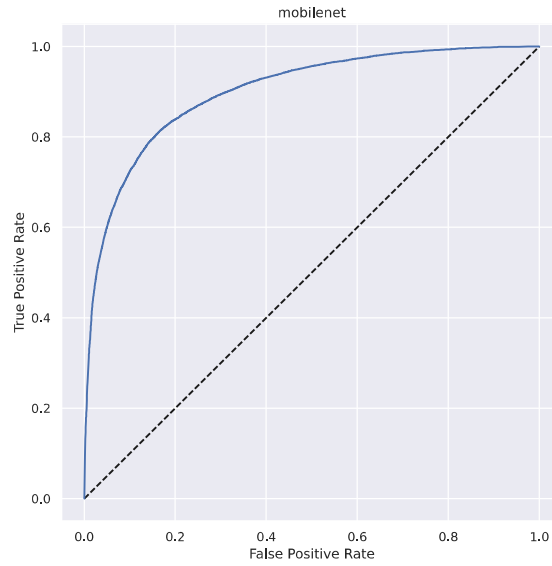


Fig. 11. The overall ROC curve for considered dataset obtained using MobileNet.

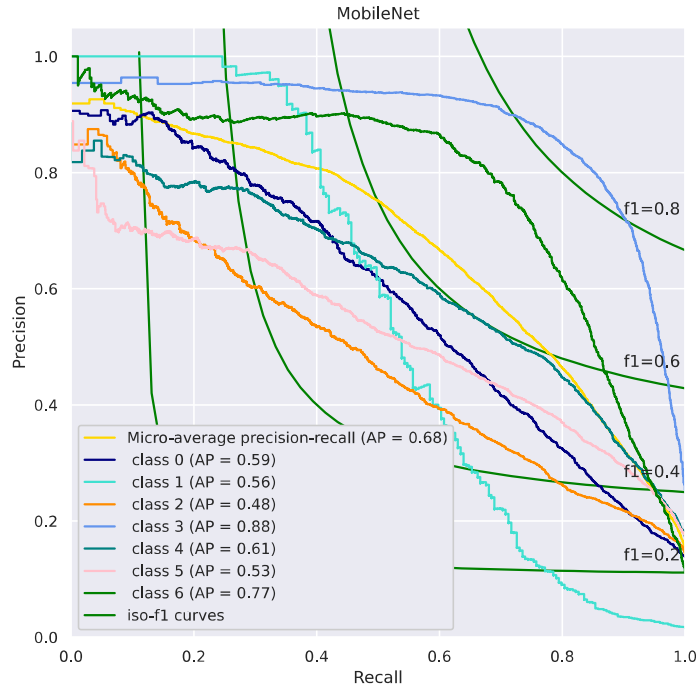


Fig. 12. Class-wise precision-recall curve for MobileNet.

6.2.4. Cost analysis

From Fig. 24, the optimal cost function over a varying range of the parameter μ is obtained. The parameters for obtaining this are considered over the range $\mu = [11; 20]$ FPS with the arrival rate of frames set to $\lambda = 10$ FPS. It was observed that the optimal cost function for our proposed approach was obtained at service rate values $\mu = 13, 14, 15$. The cost function witnessed an increase beyond that point. The surface plot in Fig. 25 illustrates the trade-off between service rate, arrival rate, and cost function, demonstrating how these variables affect the M/M/1/K queue's efficiency and costs. It can be observed that with the increase in the service rate of frames, i.e., μ , the cost function decreases; however, with a decrease in arrival rate λ of the incoming frames, the cost function $C(\lambda, \mu)$ experiences rapid growth. We can infer from the plot that the cost function becomes smaller as the service rate rises. This shows that better performance and lower costs are produced by a higher rate of service. On the other hand, when the quantity of

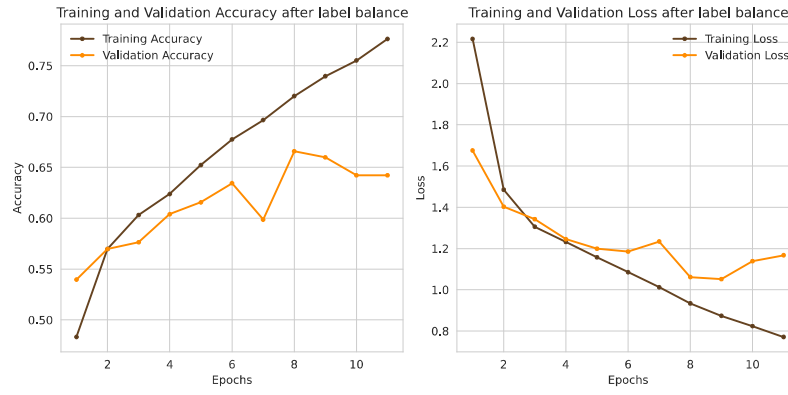


Fig. 13. Accuracy and loss curve for training and validation after “disgust” label balancing.

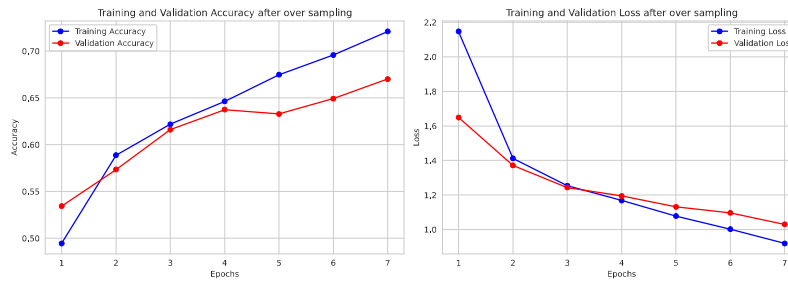


Fig. 14. Accuracy and loss curve for training and validation after large equitable labeling oversampling technique.

Table 5
Comparison of model accuracies.

Team	Model	Accuracy (%)
Raghuvanshi et al. [24]	CNN	48
Samsani et al. [25]	CNN	61.4
Hua et al. [26]	RESNET-101	58.23
Hua et al. [26]	VGG-19	62.31
Zhai et al. [27]	Net B	60.91
Zhai et al. [27]	Net B_DAL	58.33
Zhai et al. [27]	Net B_DAL_MSE	59.15
Liu et al. [29]	Single subnet	62.44
Yen et al. [28]	Finetuned Xception	44
Yen et al. [28]	Finetuned Inception	50
Yen et al. [28]	Finetuned Resnet-50	59
In this paper	Finetuned MobileNetV2	62
In this paper	Finetuned MobileNet without preprocessing	63
Liu et al. [29]	Ensemble CNN	65.03
Hua et al. [26]	CNN SUBNETWORK	66.85
Proposed	Finetuned MobileNet	67.01

incoming frames or the arrival rate of the frame grows, so does the cost function. This shows that the price of sustaining effective service levels increases as the system’s workload increases.

MobileNet is suitable for deploying FER on edge devices like the RPi due to its lightweight architecture and efficient computational abilities. MobileNet offers a good balance between model size and accuracy. Utilizing depth-wise separable convolutions significantly reduces the number of parameters compared to traditional CNN models without compromising performance. This parameter reduction is crucial for edge devices with limited memory and processing power, as it minimizes the computational burden during inference in resource-constrained IoT devices. With pre-trained weights from ImageNet, it facilitates transfer learning, enhancing performance even with limited labeled data.

Table 5 compares model accuracies across various methods for FER. The fine-tuned MobileNet outperforms other methods using different network architectures such as traditional CNNs, RESNET-101, VGG-19, as well as other fine-tuned pre-trained models like Xception, Inception, and Resnet-50, MobileNetV2 in terms of accuracy.

The results obtained from the simulations provide valuable insights into the behavior of the offloading system and highlight the relationship between the system parameters and the performance metrics of the offloading mechanism. Notably, these observations are based on the assumptions made in our modeling process. However, real-world scenarios may deviate from these assumptions, impacting the system's behavior. TPM plays a crucial role in determining the transition probabilities across various scenarios, considering the number of frames in the queue and the frames currently undergoing processing and aiding effective decision-making. Properly adjusting offloading and local processing probabilities is crucial to balance resource utilization and timely frame processing.

7. Complexity analysis

MobileNet is known for its efficiency and is designed specifically for mobile and embedded vision applications. MobileNet primarily consists of depthwise separable convolutions. In a depthwise separable convolution, each input channel undergoes convolution with its own kernel, followed by pointwise convolution to linearly combine channels, drastically reducing computations while maintaining performance. This model comprises 89 layers, including 14 Conv2D layers, 27 BatchNormalization, 27 ReLU, 13 DepthwiseConv2D, 4 ZeroPadding2D, 1 GlobalAveragePooling2D, 1 Dropout, and 1 Dense layer. The complexity of the pre-trained MobileNet model is assessed by examining operations in each layer, including factors like input and output channels, filter size, and feature map dimensions. The input layer is with a shape of (48, 48, 3), while the convolutional layer utilizes Conv2D with 32 filters, kernel size (3, 3), and ReLU activation, and then employs DepthwiseConvolution2D with 32 filters, kernel size (3, 3) initially, and ReLU activation, followed by further layers with increased filter numbers and depthwise convolutions. Additionally, batch normalization layers are utilized throughout the model. When assessing system performance and resource efficiency, we evaluate both speed and computational load, taking into account the time taken as well as the number of steps required. In neural networks where calculations like multiplications are crucial, understanding these factors optimizes systems for practical use, crucial for resource-constrained IoT devices or speed-sensitive situations, by determining the neural network's operational demands, enhancing real-world performance [57].

Multiplications_{*i*}(*M_i*) = Number of Filters × Size of the Filter × Output Size.

$$\begin{aligned} M_1 &= 32 \times (3 \times 3 \times 3) \times (24 \times 24) = 497664, M_2 = 64 \times (1 \times 1 \times 32) \times (24 \times 24) = 1179648 \\ M_3 &= 128 \times (1 \times 1 \times 64) \times (12 \times 12) = 1179648, M_4 = 128 \times (1 \times 1 \times 128) \times (12 \times 12) = 2,359,296 \\ M_5 &= 256 \times (1 \times 1 \times 128) \times (6 \times 6) = 1179648, M_6 = 256 \times (1 \times 1 \times 256) \times (6 \times 6) = 2,359,296 \\ M_7 &= 512 \times (1 \times 1 \times 256) \times (3 \times 3) = 1179648, M_8 = 512 \times (1 \times 1 \times 512) \times (3 \times 3) = 2,359,296 \\ M_9 &= 512 \times (1 \times 1 \times 512) \times (3 \times 3) = 2,359,296, M_{10} = 512 \times (1 \times 1 \times 512) \times (3 \times 3) = 2,359,296 \\ M_{11} &= 512 \times (1 \times 1 \times 512) \times (3 \times 3) = 2,359,296, M_{12} = 512 \times (1 \times 1 \times 512) \times (3 \times 3) = 2,359,296 \\ M_{13} &= 1024 \times (1 \times 1 \times 512) \times (1 \times 1) = 524288, M_{14} = 1024 \times (1 \times 1 \times 1024) \times (1 \times 1) = 524288 \end{aligned}$$

Total computations is termed as

$$\text{Total Multiplications} = \sum_{i=1}^{14} M_i,$$

where, *M_i* depicts the specific *i*th layer multiplication operations. Using *M_i*'s the total computation can be computed as

$$\begin{aligned} \text{Total Multiplications} &= 497664 + 1179648 + 1179648 + 2359296 + 1179648 + 2359296 + 1179648 + 2359296 + 2359296 \\ &+ 2359296 + 2359296 + 2359296 + 524288 + 524288 = 22779904 \end{aligned}$$

So the general expression for computation of time complexity of the convolutional layers in a network is given by:

$$T(n) = O\left(\sum_{l=1}^d n_{l-1} \cdot s_l^2 \cdot n_l \cdot m_l^2\right),$$

where *n_{l-1}* is the number of filters in the *l*-1th layer, *s_l* is the length of the filter (3 for a 3 × 3 filter size), *n_l* is the number of filters in the *l*th layer, and *m_l* is the spatial size of the output feature map [57,58]. The pooling operation and batch normalization remain constant, so they have a less significant effect on the complexity analysis. Additionally, omitting the feature map size (*m_l*) provides more flexibility in computation for the system architecture. So the time complexity of system architecture depends on the impact on filters i.e., filter sizes, and the number of filters in the spatial domain for the feature maps [58]. The time complexity *T(n)* is defined as [57,58]:

$$T(n) = O\left(\sum_{l=1}^d n_{l-1} \cdot s_l^2 \cdot n_l\right).$$

So substituting the values of parameters in the above expression, we obtained as

$$\begin{aligned} T(n) &= 3 \cdot (3^2) \cdot 32 + 32 \cdot (1^2) \cdot 64 + 64 \cdot (1^2) \cdot 128 + 128 \cdot (1^2) \cdot 128 + 128 \cdot (1^2) \cdot 256 + 256 \cdot (1^2) \cdot 256 + 256 \cdot (1^2) \cdot 512 + 512 \\ &\cdot (1^2) \cdot 512 + 512 \cdot (1^2) \cdot 512 + 512 \cdot (1^2) \cdot 512 + 512 \cdot (1^2) \cdot 512 + 512 \cdot (1^2) \cdot 512 + 512 \cdot (1^2) \cdot 512 + 512 \cdot (1^2) \cdot 1024 + 1024 \cdot (1^2) \cdot 1024 \end{aligned}$$

So the time complexity for the 14 convolution is *T(n)* = 3140448.

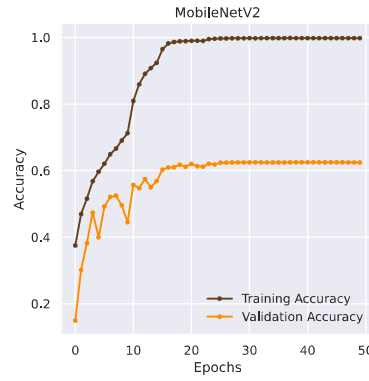


Fig. 15. Comparison of training and validation accuracy for varying number of epochs for MobileNetV2.

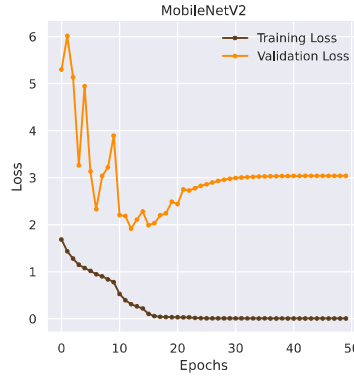


Fig. 16. Comparison of training and validation loss for varying number of epochs for MobileNetV2.

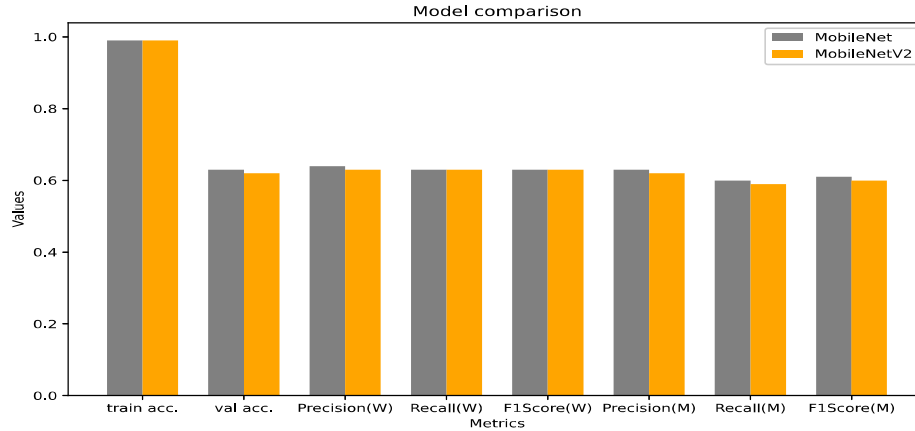


Figure 17: MobileNet vs MobilenetV2

Fig. 17. MobileNet vs MobilenetV2.

Furthermore, the inference time plays a crucial role in processing a single image. The testing results are reported to be approximately 23.75 ms for averages of real-time raw images. Some samples of raw testing are depicted in Fig. 19. This rapid processing time demonstrates the efficiency of the implemented pipeline, making it suitable for real-time applications such as emotion recognition in video streams.

Table 6 compares the inference times (in milliseconds) of different models, showing that the proposed finetuned MobileNet achieves the fastest inference time of 23.75 ms, outperforming both ResNet50 [59] and MobileNetV2 pretrained models on raw images in terms of the speed of DTL. We have presented the accuracy and loss curves for MobileNet V2 in Fig. 15, and 16, respectively. Furthermore, we have compared MobileNet with MobileNet V2 in terms of classification performance in Fig. 17. The

Table 6
Inference time comparison.

Architecture	Single image inference Time (ms)
ResNet50 pretrained [59]	35.9 ms
MobileNetV2 pretrained	27.19 ms
Finetuned MobileNet (Proposed)	23.75 ms

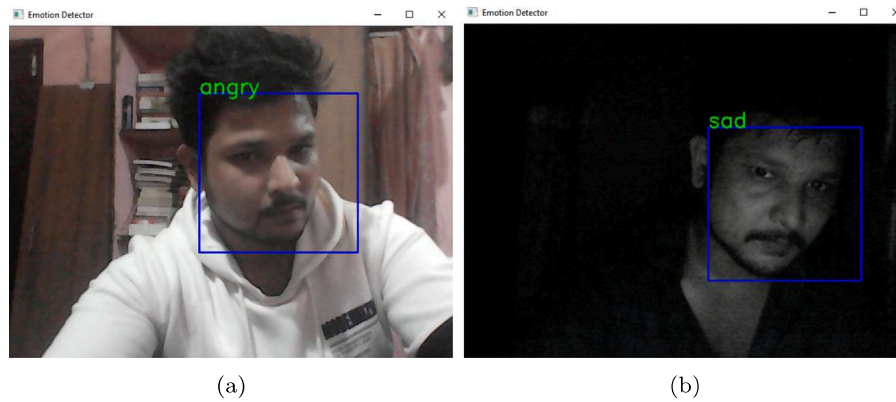


Fig. 18. Real-time facial emotion extraction: (a) Anger, (b) Sadness. Demonstrating system effectiveness in challenging lighting condition, emphasizing night vision detection.

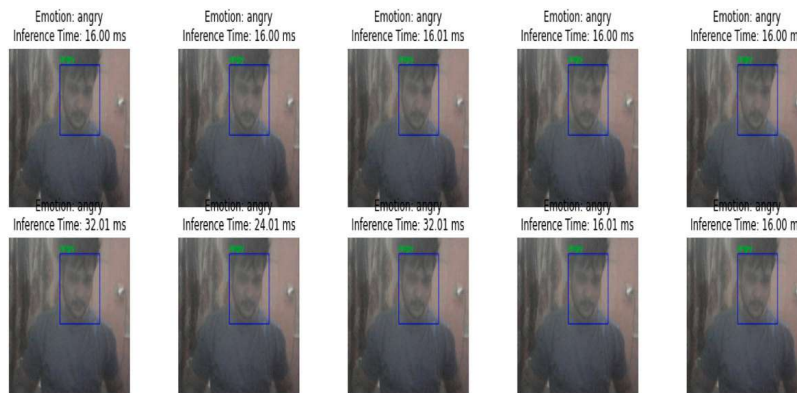


Fig. 19. Samples of real-time single raw image testing with inference time within very short time.

results indicate that the proposed finetuned MobileNet demonstrates superior efficiency, outperforming MobileNetV2 in terms of speed and accuracy.

Table 7 provides a comparison between the Fog-node and Mist-node models in terms of size, inference time, and performance metrics. The Mist-node model (TF.lite) has a significantly smaller size (12.23 MB) compared to the Fog-node (Keras h5.) model (37.30 MB) and also achieves a faster inference time of 7.58 ms, in contrast to 23.75 ms for the Fog-node. Despite the smaller size, the Mist-node model achieves a reasonable level of accuracy, with a testing score of 22/40 on raw images. Fig. 20 depicts the confusion matrix of raw image testing for the Mist model. This performance makes it a good option for deployment in scenarios where computational resources and time efficiency are critical factors.

Fig. 26 shows some testing results with AffectNet dataset samples [60] and other raw images using the Mist model. The correct classification is shown in (a). In (b), sadness is misclassified as fear in raw images. Happy is misclassified as neutral in (c). In (d) and (e), multiple people are correctly predicted. Figs. 27 (a), (b), and (c) show real-time webcam testing using the Mist model. Figs. 28 (a) and (b) compare the performance of Mist and Fog models on DSLR raw image, focusing on inference time, where Mist outperforms Fog.

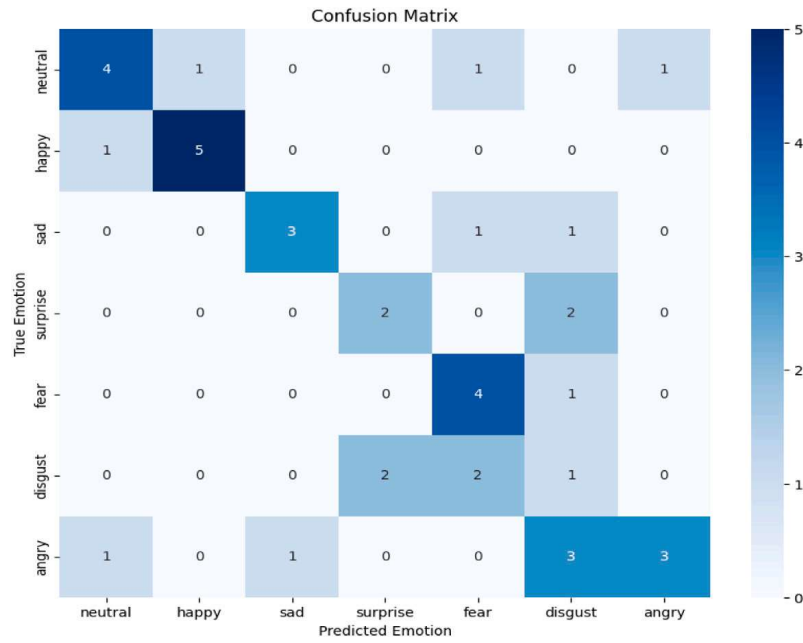


Fig. 20. Testing raw images using Light-weight model.

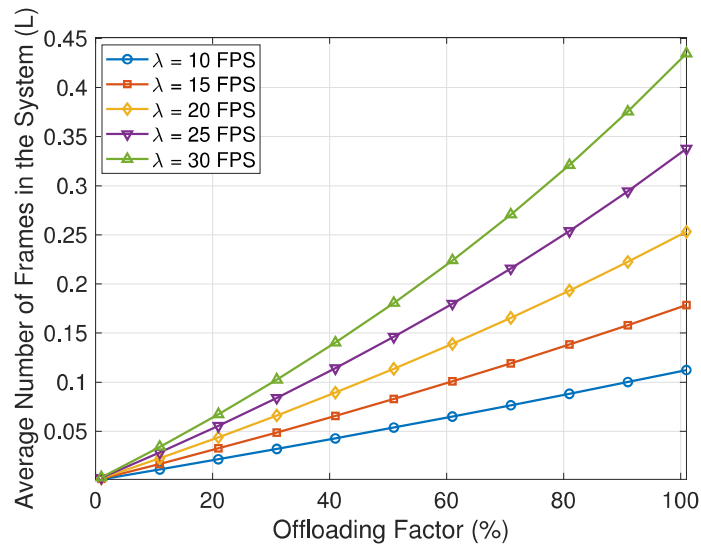
Fig. 21. Comparison of Average Number of Frames in the System (L) with Offloading Factor (%) for different values of λ .

Table 7

Comparison of Fog-node and Mist-node Model.

Property	Fog-node	Mist-node
Model Size (MB)	37.30	12.23
Inference Time (ms)	23.75	7.58(raw),16.42(cam)
Validation/testing	63% (dataset val acc.)	22/40 (raw testing score)

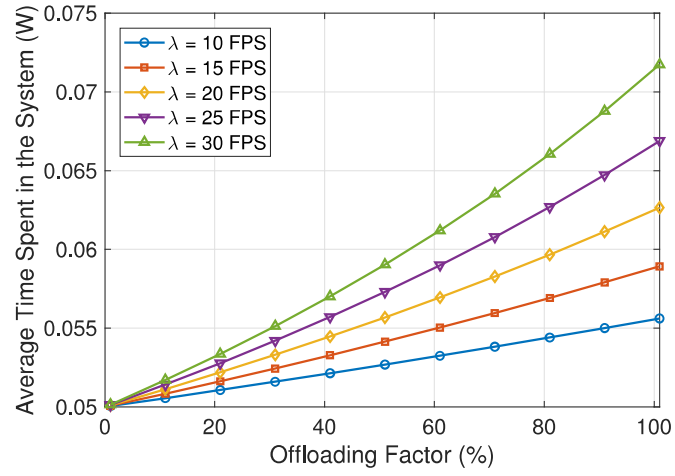


Fig. 22. Comparison of Average Time Spent in the System (W) with Offloading Factor (%) for different values of λ .

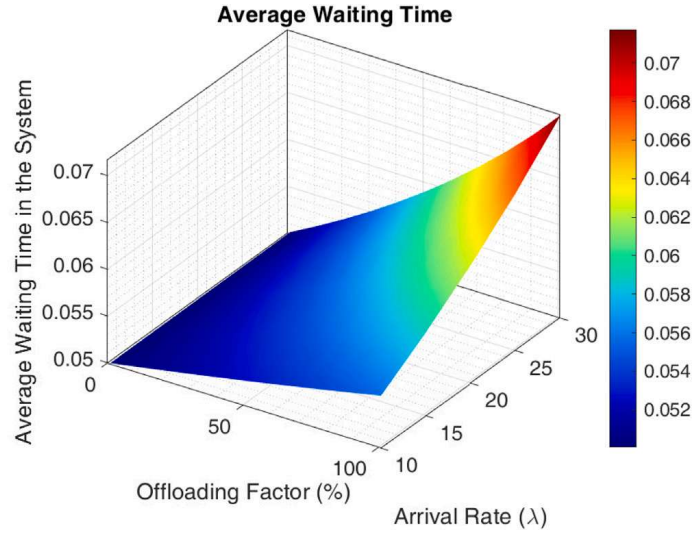


Fig. 23. Trade-off between Average Waiting Time in the System with Offloading Factor (%) and Arrival rates λ .

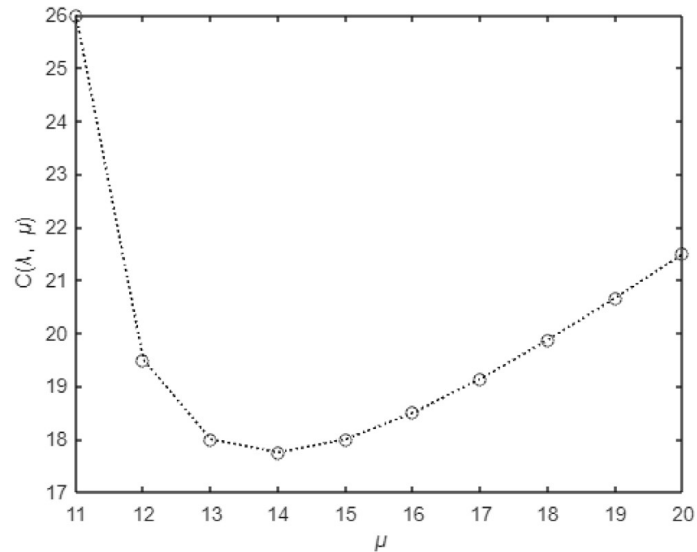


Fig. 24. Estimated cost function over different service rates $\mu = [11; 20]$ FPS and arrival rate $\lambda = 10$ FPS.

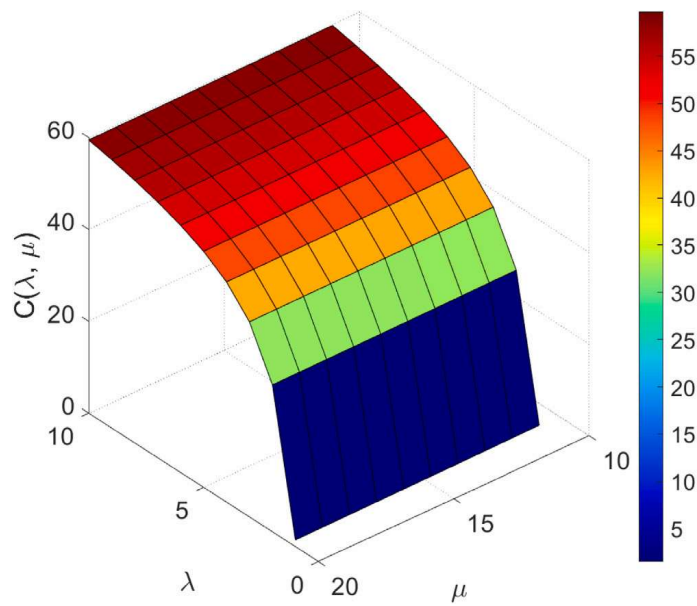
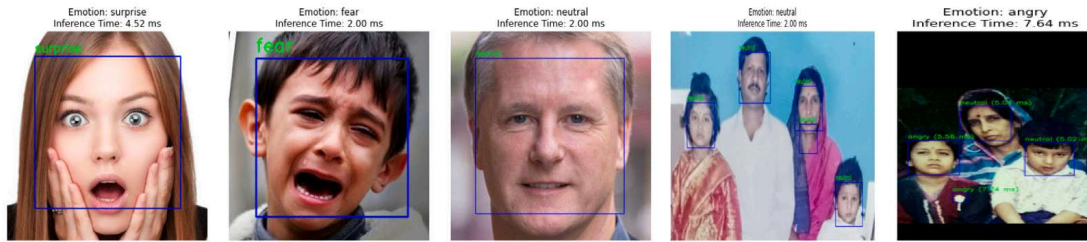
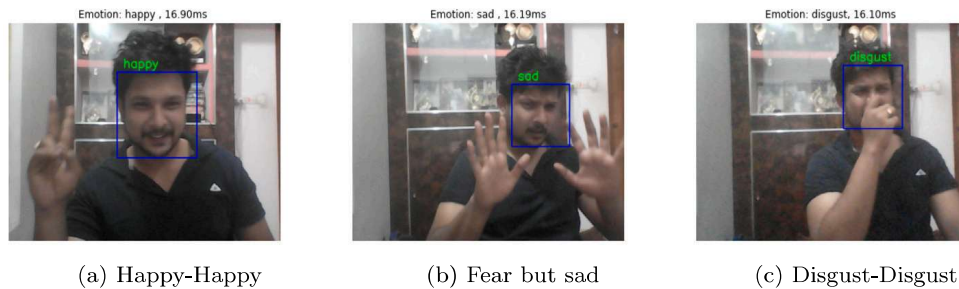


Fig. 25. Performance trade-off between λ and μ for cost function $C(\lambda, \mu)$.



(a) Surprise (b) Sad but fear (c) hap.. but neu. (d) Multiple person (e) Multiple person

Fig. 26. Testing results of some correct predictions and misclassifications on raw input images using Mist model.



(a) Happy-Happy (b) Fear but sad (c) Disgust-Disgust

Fig. 27. Realtime testing results using Mist model.



(a) Testing with Fog model (b) Testing with Mist model

Fig. 28. DSLR picture inference time comparison between Mist and Fog models.

8. Conclusion

Several industries have used FER, demonstrating its adaptability and potential. One particular area of use is in the domain of human–computer interaction, where integrating FER with IoT technologies can significantly enhance user experiences by enabling systems to dynamically adjust in response to people’s emotional states. FER’s requirement to use DTL has been driven by the increased demand for precise and quick emotion analysis across many applications. In this study, we devised a cutting-edge Mist-Fog architecture for IoT-enabled real-time FER in smart city 4.0. Our suggested method makes use of MobileNet, a DTL technique that enables quick and effective image classification on embedded and mobile devices. We focused on the difficulties of running deep neural networks within available power and computing constraints. The Mobile-Net approach drastically lowers the computational costs while maintaining respectable accuracy levels using depth-wise separable convolutions. The model achieved an overall ROC-AUC score of 88% on the FER-2013 dataset. We created a fog-based architecture with a mist layer comprising RPi IoT edge devices to simulate a smart city scenario. In order to test the performance of the system, we also integrated the M/M/1/K queuing system, which uses the FCFS scheduling criteria and follows an MMPP arrival rate. Finally, the numerical outcomes pertaining to the average number of frames in the system, average time spent, offloading factor, and processing cost for the proposed Mist-Fog architecture were presented subject to different system parameters. Finally, we compare our work with existing models, demonstrating that our model outperforms them in terms of accuracy while imposing a lighter computational load and achieving greater speed.

Future work will focus on implementing high-performance IoT edge devices for FER and validating the efficacy of our suggested approach over different benchmark strategies. We aimed to deploy the implemented system in natural urban environments to observe its performance under real-life conditions, identify practical challenges, and enable continuous monitoring for workload management in IoT platforms, with large-scale hardware support for multi-server concept for offloading frames from the mist layer and also study the overall time performance by employing different models. We will further consider improving the FER system by exploring more sophisticated DTL methods on various FER datasets compatible with the experimented edge interface.

CRedit authorship contribution statement

Aditya Ranjan Dalabehera: Validation, Investigation, Data curation. **Sujit Bebertta:** Writing – original draft, Methodology, Investigation, Formal analysis, Conceptualization. **Naveen Kumar:** Writing – review & editing, Validation, Supervision, Project administration. **Dilip Senapati:** Writing – review & editing, Validation, Supervision, Project administration, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This research is supported in part by Science and Technology Department, Government of Odisha (Grant No.: ST-SCSTMISC-0045-2021), Higher Education Department, Government of Odisha under OHEPEE (Gant no. HE-PTC-WB-02017) and Faculty Research Programme (FRP) of the IoE scheme, University of Delhi (Grant No.: IoE/2023-24/12/FRP).

References

- [1] Sujit Bebertta, Amit Kumar Singh, Bibudhendu Pati, Dilip Senapati, A robust energy optimization and data reduction scheme for IoT-based indoor environments using a local processing framework, *J. Netw. Syst. Manage.* 29 (2021) 1–28.
- [2] Hemant Kumar Apat, Rashmiranjan Nayak, Bibhudatta Sahoo, A comprehensive review on internet of things application placement in fog computing environment, *Internet Things* (2023) 100866.
- [3] Kun Zhang, Yu Zhou, Chaoyang Wang, Haizhuang Hong, Jing Chen, Qian Gao, Mostafa Ghobaei-Arani, Towards an automatic deployment model of IoT services in fog computing using an adaptive differential evolution algorithm, *Internet Things* 24 (2023) 100918.
- [4] Sujit Bebertta, Aditya Ranjan Dalabehera, Bibudhendu Pati, Chhabi Rani Panigrahi, Gyana Ranjan Nanda, Biswajit Sahu, Dilip Senapati, An intelligent spatial stream processing framework for digital forensics amid the covid-19 outbreak, *Smart Health* 26 (2022) 100308.
- [5] A. Saxena, A. Khanna, D. Gupta, Emotion recognition and detection methods: A comprehensive survey, *J. Artif. Intell. Syst.* 2 (1) (2020) 53–79.
- [6] G. Muhammad, M.S. Hossain, Emotion recognition for cognitive edge computing using deep learning, *IEEE Internet Things J.* 8 (23) (2021) 16894–16901.
- [7] C. Xu, C. Yan, M. Jiang, F. Alenezi, A. Alhudaif, N. Alnaim, K. Polat, W. Wu, A novel facial emotion recognition method for stress inference of facial nerve paralysis patients, *Expert Syst. Appl.* 197 (2022) 116705.
- [8] Aditya Ranjan Dalabehera, Sujit Bebertta, Amit Kumar Singh, Dilip Senapati, Real-time criticality evaluation through vision-based human-centric emotion, activity and object interactions, in: *2024 1st International Conference on Cognitive, Green and Ubiquitous Computing, IC-CGU, IEEE, 2024*, pp. 1–6.
- [9] M. Kalpana Chowdary, Tu N. Nguyen, D. Jude Hemanth, Deep learning-based facial emotion recognition for human–computer interaction applications, *Neural Comput. Appl.* (2021) 1–18.

- [10] Dimin Zhu, Yuxi Fu, Xinjie Zhao, Xin Wang, Hanxi Yi, Facial emotion recognition using a novel fusion of convolutional neural network and local binary pattern in crime investigation, 2022.
- [11] Ashraf Abbas M. Al-Modwahi, Onkemetse Sebetela, Lefoko Nehemiah Batleng, Behrang Parhizkar, Arash Habibi Lashkari, Facial expression recognition intelligent security system for real time surveillance, in: Proc. of World Congress in Computer Science, Computer Engineering, and Applied Computing, 2012.
- [12] Felipe Zago Canal, Tobias Rossi Müller, Jhennifer Cristine Matias, Gustavo Gino Scotton, Antonio Reis de Sa Junior, Eliane Pozzebon, Antonio Carlos Sobieranski, A survey on facial emotion recognition techniques: A state-of-the-art literature review, *Inform. Sci.* 582 (2022) 593–617.
- [13] Suchitra Saxena, Shikha Tripathi, T.S.B. Sudarshan, An intelligent facial expression recognition system with emotion intensity classification, *Cogn. Syst. Res.* 74 (2022) 39–52.
- [14] M.A.H. Akhand, Shuvendu Roy, Nazmul Siddique, Md Abdus Samad Kamal, Tetsuya Shimamura, Facial emotion recognition using transfer learning in the deep cnn, *Electronics* 10 (9) (2021) 1036.
- [15] Harishchandra Dubey, Admir Monteiro, Nicholas Constant, Mohammadreza Abtahi, Debanjan Borthakur, Leslie Mahler, Yan Sun, Qing Yang, Umer Akbar, Kunal Mankodiya, Fog computing in medical internet-of-things: architecture, implementation, and applications, in: Handbook of Large-Scale Distributed Computing in Smart Healthcare, 2017, pp. 281–321.
- [16] Soumya Kanti Datta, Christian Bonnet, Jerome Haerri, Fog computing architecture to enable consumer centric internet of things services, in: 2015 International Symposium on Consumer Electronics, ISCE, IEEE, 2015, pp. 1–2.
- [17] Mohammad Aazam, Sherali Zeadally, Khaled A. Harras, Deploying fog computing in industrial internet of things and industry 4.0, *IEEE Trans. Ind. Inform.* 14 (10) (2018) 4674–4682.
- [18] Mohd Nadhir Ab Wahab, Amril Nazir, Anthony Tan Zhen Ren, Mohd Halim Mohd Noor, Muhammad Firdaus Akbar, Ahmad Sufiril Azlan Mohamed, Efficientnet-lite and hybrid cnn-knn implementation for facial expression recognition on raspberry pi, *IEEE Access* 9 (2021) 134065–134080.
- [19] Sujit Bebortta, Dilip Senapati, Chhabi Rani Panigrahi, Bibudhendu Pati, An adaptive modeling and performance evaluation framework for edge-enabled green iot systems, *IEEE Trans. Green Commun. Netw.* 6 (2) (2021) 836–844.
- [20] Hongyan Yu, Qiyuan Wang, Songtao Guo, Energy-efficient task offloading and resource scheduling for mobile edge computing, in: 2018 IEEE International Conference on Networking, Architecture and Storage, NAS, 2018, pp. 1–4.
- [21] Bo Tang, Zhen Chen, Gerald Hefferman, Tao Wei, Haibo He, Qing Yang, A hierarchical distributed fog computing architecture for big data analysis in smart cities, in: Proceedings of the ASE BigData & SocialInformatics 2015, 2015, pp. 1–6.
- [22] P. Vijayakumari, P.G. Kuppusamy, Eethamakula Kosalendra, K. Krishnamoorthi, S. Diwakaran, Real-time scanning and tracking of health records through a centralized ecosystem based on internet of things and fog computing, in: 2023 International Conference on Artificial Intelligence and Knowledge Discovery in Concurrent Engineering, ICECONF, IEEE, 2023, pp. 1–8.
- [23] Muhammad Sajjad, Mansoor Nasir, Fath U. Min Ullah, Khan Muhammad, Arun Kumar Sangaiah, Sung Wook Baik, Raspberry pi assisted facial expression recognition framework for smart security in law-enforcement services, *Inform. Sci.* 479 (2019) 416–431.
- [24] A. Raghuvanshi, V. Choksi, Facial expression recognition with convolutional neural networks, in: CS231n Course Projects, Vol. 362, 2016.
- [25] S. Samsani, V.A. Gottala, A real-time automatic human facial expression recognition system using deep neural networks, in: Information and Communication Technology for Sustainable Development: Proceedings of ICT4SD 2018, Springer, 2020, pp. 431–441.
- [26] W. Hua, F. Dai, L. Huang, J. Xiong, G. Gui, HERO: Human emotions recognition for realizing intelligent Internet of Things, *IEEE Access* 7 (2019) 24321–24332.
- [27] Yikui Zhai, Jian Liu, Junying Zeng, Vincenzo Piuri, Fabio Scotti, Zilu Ying, Ying Xu, Junying Gan, Deep convolutional neural network for facial expression recognition.
- [28] Chih-Ta Yen, Kang-Hua Li, Discussions of different deep transfer learning models for emotion recognitions, *IEEE Access* 10 (2022) 102860–102875, <http://dx.doi.org/10.1109/ACCESS.2022.3209813>; Image and Graphics: 9th International Conference, ICGI 2017, Shanghai, China, September 13–15, 2017, Revised Selected Papers, Part 1 9, Springer, 2017, pp. 211–223.
- [29] Kuang Liu, Mingmin Zhang, Zhigeng Pan, Facial expression recognition with CNN ensemble, in: 2016 International Conference on Cyberworlds, CW, IEEE, 2016, pp. 163–166.
- [30] Palasatti Srinivasa Reddi, A. Sri Krishna, Cnn implementing transfer learning for facial emotion recognition, *Int. J. Intell. Syst. Appl. Eng.* 11 (4s) (2023) 35–45.
- [31] Dina Shehada, Ayad Turkey, Wasiq Khan, Bilal Khan, Abir Hussain, A lightweight facial emotion recognition system using partial transfer learning for visually impaired people, *IEEE Access* 11 (2023) 36961–36969.
- [32] Venkatesh Srinivasan, Sascha Meudt, Friedhelm Schwenker, Deep learning algorithms for emotion recognition on low power single board computers, in: Multimodal Pattern Recognition of Social Signals in Human-Computer-Interaction: 5th IAPR TC 9 Workshop, MPRSS 2018, Beijing, China, August 20 2018, Revised Selected Papers 5, Springer, 2019, pp. 59–70.
- [33] Gaurav Meena, Krishna Kumar Mohbey, Sunil Kumar, Rahul Kumar Chawda, Sandeep V. Gaikwad, Image-based sentiment analysis using inceptionv3 transfer learning approach, *SN Comput. Sci.* 4 (3) (2023) 242.
- [34] Sudheer Babu Punuri, Sanjay Kumar Kuanar, Manjur Kolhar, Tusar Kanti Mishra, Abdalla Alameen, Hitesh Mohapatra, Soumya Ranjan Mishra, Efficient net-xgboost: an implementation for facial emotion recognition using transfer learning, *Mathematics* 11 (3) (2023) 776.
- [35] Dezheng H. Abdulazeez, Shavan K. Askar, Offloading mechanisms based on reinforcement learning and deep learning algorithms in the fog computing environment: A comprehensive review, *IEEE Access* (2023).
- [36] Gaurav Goel, Rajeev Tiwari, Resource scheduling techniques for optimal quality of service in fog computing environment: A review, *Wirel. Pers. Commun.* (2023) 1–24.
- [37] Mehdi Hosseinzadeh, Elham Azhir, Jan Lansky, Stanislava Mildeova, Omed Hassan Ahmed, Mazhar Hussain Malik, Faheem Khan, Task scheduling mechanisms for fog computing: A systematic survey, *IEEE Access* (2023).
- [38] Mati B. Terefe, Heezin Lee, Nojung Heo, Geoffrey C. Fox, Sangyoon Oh, Energy-efficient multisite offloading policy using markov decision process for mobile cloud computing, *Pervasive Mob. Comput.* 27 (2016) 75–89.
- [39] Ian J. Goodfellow, Dumitru Erhan, Pierre Luc Carrier, Aaron Courville, Mehdi Mirza, Ben Hamner, Will Cukierski, Yichuan Tang, David Thaler, Dong-Hyun Lee, Yingbo Zhou, Chetan Ramaiah, Fangxiang Feng, Ruifan Li, Xiaojie Wang, Dimitris Athanasakis, John Shawe-Taylor, Maxim Milakov, John Park, Radu Ionescu, Marius Popescu, Cristian Grozea, James Bergstra, Jingjing Xie, Lukasz Romaszko, Bing Xu, Zhang Chuang, Yoshua Bengio, Challenges in representation learning: A report on three machine learning contests, in: Minho Lee, Akira Hirose, Zeng-Guang Hou, Rhee Man Kil (Eds.), Neural Information Processing, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, pp. 117–124.
- [40] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, Hartwig Adam, Mobilenets: Efficient convolutional neural networks for mobile vision applications, 2017, arXiv preprint [arXiv:1704.04861](https://arxiv.org/abs/1704.04861).
- [41] Behnam Asadi, Hui Jiang, On approximation capabilities of relu activation and softmax output layer in neural networks, 2020, arXiv preprint [arXiv:2002.04060](https://arxiv.org/abs/2002.04060).
- [42] Pushparaja Murugan, Implementation of deep convolutional neural network in multi-class categorical image classification, 2018, arXiv preprint [arXiv:1801.01397](https://arxiv.org/abs/1801.01397).
- [43] Diederik P. Kingma, Jimmy Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint [arXiv:1412.6980](https://arxiv.org/abs/1412.6980).

- [44] P. McCullagh, Face detection by using haar cascade classifier, *Wasit J. Comput. Math. Sci.* 2 (1) (2023) 1–5.
- [45] M. Monica Dhana Ranjini, M. Paul Jeyaraj, M. Senthil Kumar, T. Arun Prasath, G. Prabhakar, Haar cascade classifier-based real-time face recognition and face detection, in: 2023 4th International Conference on Electronics and Sustainable Communication Systems, ICESC, 2023, pp. 990–995.
- [46] Sujit Bebortta, Dilip Senapati, Chhabi Rani Panigrahi, Bibudhendu Pati, Adaptive performance modeling framework for qos-aware offloading in mec-based iiot systems, *IEEE Internet Things J.* 9 (12) (2021) 10162–10171.
- [47] Songtao Guo, Bin Xiao, Yuanyuan Yang, Yang Yang, Energy-efficient dynamic offloading and resource scheduling in mobile cloud computing, in: IEEE INFOCOM 2016-the 35th Annual IEEE International Conference on Computer Communications, IEEE, 2016, pp. 1–9.
- [48] Thinh Quang Dinh, Jianhua Tang, Quang Duy La, Tony Q.S. Quek, Offloading in mobile edge computing: Task allocation and computational frequency scaling, *IEEE Trans. Commun.* 65 (8) (2017) 3571–3584.
- [49] Yangzhe Liao, Liqing Shou, Quan Yu, Qingsong Ai, Quan Liu, Joint offloading decision and resource allocation for mobile edge computing enabled networks, *Comput. Commun.* 154 (2020) 361–369.
- [50] Munatel Mohammed, Abdelkrim Haqiq, Dynamic resource allocation for service in mobile cloud computing with markov modulated arrivals, *Int. J. Model. Simul. Sci. Comput.* 12 (05) (2021) 2150038.
- [51] Lluís Mas, Jordi Vilaplana, Jordi Mateo, Francesc Solsona, A queuing theory model for fog computing, *J. Supercomput.* 78 (8) (2022) 11138–11155.
- [52] Zheng Chang, Zhenyu Zhou, Tapani Ristaniemi, Zhisheng Niu, Energy efficient optimization for computation offloading in fog computing system, in: GLOBECOM 2017-2017 IEEE Global Communications Conference, 2017, pp. 1–6.
- [53] Donald Gross, *Fundamentals of Queueing Theory*, John Wiley & sons, 2008.
- [54] Sujit Bebortta, Dilip Senapati, Toward cost-aware computation offloading in iot-based mec systems, *Nat. Acad. Sci. Lett.* (2023) 1–4.
- [55] Sujit Bebortta, Om Pal, Dilip Senapati, Subhranshu Sekhar Tripathy, Cost-aware computation offloading for managing cloud-bursts in IoT-based cloud-fog networks, in: 2024 1st International Conference on Cognitive, Green and Ubiquitous Computing, IC-CGU, IEEE, 2024, pp. 1–6.
- [56] Margherita Grandini, Enrico Bagli, Giorgio Visani, Metrics for multi-class classification: an overview, 2020, arXiv preprint arXiv:2008.05756.
- [57] Shruti Jaiswal, Gora Chand Nandi, Robust real-time emotion detection system using CNN architecture, *Neural Comput. Appl.* 32 (15) (2020) 11253–11262.
- [58] Kaiming He, Jian Sun, Convolutional neural networks at constrained time cost, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 5353–5360.
- [59] George Cosmin Porusniuc, Florin Leon, Radu Timofte, Casian Miron, Convolutional neural networks architectures for facial expression recognition, in: 2019 E-Health and Bioengineering Conference, EHB, IEEE, 2019, pp. 1–6.
- [60] A. Mollahosseini, B. Hasani, M.H. Mahoor, AffectNet: A database for facial expression, valence, and arousal computing in the wild, *IEEE Trans. Affect. Comput.* 10 (1) (2019) 18–31.